

TransitDuet: bi-level reinforcement learning for joint bus target-headway planning and holding control

Abstract

We present TransitDuet, a bi-level reinforcement learning framework that jointly optimises adaptive bus target-headway planning and real-time holding control. This joint problem is fundamentally harder than either task in isolation: the planning and control layers operate on different timescales, observe different state spaces, and interact through delayed, asynchronous feedback—properties that violate the synchronous co-execution assumption underlying existing hierarchical reinforcement learning methods. TransitDuet addresses this through three mechanisms. First, a goal-conditioned cross-level coupling in the spirit of HIRO — adopted, not re-derived, and selected via end-to-end comparison against a literal cross-advantage augmentation and a three-channel launch-time-perturbation alternative (Section V): the upper outputs a per-dispatch target-headway shift, and the lower’s Lagrangian cost is computed against the resulting goal headway, with no upper-advantage injection into the lower’s transitions. This avoids the covariate shift that destabilises a literal cross-advantage augmentation while keeping the upper–lower interaction unidirectional and stable. Second, a measurement-driven online gradient descent (θ -OGD) on the upper-reward weights adaptively penalises fleet overshoot — this is a soft adaptive penalty rather than a strict policy projection, but it removes the need for a hand-tuned overshoot coefficient and converges to a regime where the constraint is rarely violated in expectation. Third, Lagrangian relaxation within the lower policy penalizes headway deviations, converting a hard service-regularity requirement into a soft dual objective. We evaluate TransitDuet in a calibrated microsimulation of a high-frequency bus corridor with stochastic demand and traffic. A complementary technique, target-policy-corrected lower SAC (TPC), uses importance-reweighting to align the lower’s training distribution with its eventual deployment distribution; combined with the goal-conditioned coupling and trained for 300 episodes per seed (search baselines: 320 episodes), TransitDuet attains the best numerical mean of any method on all four reported metrics (wait, CV, overshoot, composite) under a near-unified, approximately budget-matched evaluation protocol; the overshoot margin over CMA-ES is 0.03 and we treat it as within seed noise rather than as a substantive advantage. Across three random seeds and an elastic fleet budget sampled per episode from [8, 16], TransitDuet reduces composite cost by 8.2% (1.418 vs 1.544) relative to the strongest hand-tuned fixed timetable, 4.7% relative to the strongest search baseline (CMA-ES at 1.488), and is markedly more reliable than GA, which has a 1-in-3 cold-start failure mode that pushes its mean composite to 1.800. Seed variance on composite cost is the lowest of any method (std 0.016, vs 0.022 for CMA-ES, 0.040 for Fixed, and 0.458 for GA), and a single trained policy per seed produces an operational Pareto frontier across the nine fleet sizes $N_{\text{fleet}} \in [8, 16]$ (composite-knee at $N_{\text{fleet}}=14$, no retraining required). Generalisation degrades gracefully: wait rises by 16% at $\sigma_{\text{route}}=2.0$ and 27% at $\sigma_{\text{route}}=3.0$ (double the training stochasticity). We acknowledge the small seed count ($n=3$) limits significance testing; we report ranges and standard deviations throughout and avoid pairwise significance claims.

Keywords: bus holding control; target-headway planning; hierarchical reinforcement learning; multi-timescale control; constrained optimization.

1 Introduction

High-frequency bus systems face a persistent coordination failure. Timetables are planned offline to balance fleet utilization against passenger demand, yet the headways they prescribe degrade within

minutes once buses encounter stochastic traffic and boarding delays. Real-time holding control can restore regularity, but only within the headway structure it inherits. When the timetable itself is poorly matched to demand—dispatching too few buses in the peak or too many in the off-peak—no amount of holding can recover acceptable service. Planning and control are thus coupled: the quality of one layer bounds the achievable performance of the other.

Despite this coupling, existing work treats the two problems separately. The timetable optimization literature formulates headway or frequency design as mixed-integer programs solved offline [1], while the holding control literature trains reinforcement learning (RL) agents that take the timetable as given [2]. Joint optimization would seem a natural application of hierarchical RL, yet three gaps prevent a straightforward application.

Asynchrony between planning and control.

Hierarchical RL frameworks such as the options framework [3] and goal-conditioned methods like HIRO [4] assume that the upper and lower levels share a common state clock or that the upper level sets subgoals consumed by the lower level within a fixed horizon. In bus operations, the upper policy acts at dispatch events (once per bus departure), while the lower policy acts at every station arrival across all active buses. The two levels observe different state spaces—the upper level sees route-level demand forecasts and fleet status, whereas the lower level sees per-bus headway deviations and passenger loads—and interact through delayed feedback spanning an entire trip lifecycle. No shared clock or fixed subgoal horizon exists.

Cross-advantage under asynchrony.

Pan et al. [5] recently introduced cross-advantage augmentation for bi-level robot locomotion, where an upper style policy and a lower tracking policy execute synchronously on the same state. Their augmented advantage $\hat{A}_U = A_U + \beta A_L$ directly sums the two advantages at each time step. This elegant mechanism breaks down when the two levels operate on different timescales with different state and action spaces: there is no single time step at which A_U and A_L are jointly defined. Generalising cross-level coupling to the asynchronous, multi-timescale setting requires propagating execution outcomes (rather than per-step advantages) over the variable-length trip lifecycle initiated by each dispatch, and exposing lower-level holding statistics to the upper policy through state augmentation rather than reward backflow.

Fleet and regularity constraints.

Bus operations impose feasibility-style constraints—a fleet-size budget N_{fleet} and bounded headway deviation—that naive reward shaping handles poorly. Constrained RL methods such as RCPO [6] address single-level constraint satisfaction but do not account for constraints that span a bi-level structure, where the upper level controls resource allocation and the lower level controls service regularity.

We propose TransitDuet, a bi-level RL framework that addresses all three gaps. The upper policy π_U outputs a per-dispatch target-headway shift $\delta_t \in [-120, 120]$ s at each dispatch event; the lower’s headway target for that trip is reset to $h^{\text{target}} + \delta_t$. The lower policy π_L , a soft actor-critic with Lagrangian penalties [7], selects holding times at every station arrival with parameter sharing across buses, and its constraint is computed against the dispatch-specific goal headway. This goal-conditioned coupling, in the spirit of HIRO [4], replaces the literal cross-advantage augmentation of RoboDuet [5] (Section 4.4); two auxiliary upper-only signals (a holding-feedback state channel and a per-dispatch hindsight credit on the upper reward) provide credit assignment without injecting upper signals into the lower’s transitions. Lower-policy training begins with a 30-episode warmup during which the upper is held at $\delta_t = 0$, after which the goal channel engages. Fleet-size feasibility is enforced by a θ -weighted online gradient descent projection on the measurement vector $\mathbf{z}$ of dispatched buses, inspired by the ApproPO framework [8], while headway uniformity enters the lower-level objective through Lagrangian relaxation.

Contributions.

The specific contributions of this paper are:

1. Goal-conditioned cross-level coupling. We adopt a HIRO-style goal-conditioned coupling [4]:

the upper outputs a per-dispatch target-headway shift, and the lower’s Lagrangian cost is computed against the resulting goal headway, with no upper-advantage injection into the lower’s transitions. We compare this end-to-end against (i) a literal cross-advantage augmentation following HAAR [9] with a PIPER-style reachability gate [10], and (ii) a three-channel coupling that uses δ_t as a launch-time perturbation with hindsight reward credit; the goal-conditioned design has the best mean composite cost (1.418 vs 1.496 vs 1.521) and the lowest seed variance ($\geq 5\times$ tighter than the alternatives) on the same evaluation protocol.

2. Joint target-headway–holding formulation. We cast adaptive bus target-headway planning and holding control as a bi-level Markov decision process with heterogeneous state-action spaces and asynchronous transitions; the upper level sets per-dispatch target headways atop a fixed nominal timetable rather than redesigning frequencies offline, providing a formal problem structure absent from prior bus RL work.
3. Adaptive constraint handling via θ -OGD and Lagrangian relaxation. We adaptively penalise fleet overshoot at the upper level via measurement-driven OGD on the reward weights and headway deviation at the lower level via a dual variable. Both are soft constraints in the sense that they shape the reward rather than project the policy; the OGD scheme removes the need to hand-tune an overshoot coefficient (as a reward-shaping baseline would require) but does not provide a worst-case feasibility guarantee.
4. Target-policy-corrected lower SAC (TPC). We identify a covariate-shift failure mode of joint bi-level RL on tightly coupled control: the lower policy is trained alongside an exploring upper policy, biasing the lower’s training distribution away from deployment. We address this via importance-weighted lower SAC against an EMA “deployment” upper policy, with validation-based checkpoint selection. Under the goal-conditioned coupling we ultimately adopt, the upper modifies the lower’s target rather than the distribution of states it sees, so TPC’s in-distribution effect is small (section 5.3); its main practical value is on the launch-time-perturbation coupling we tested as an alternative.
5. Empirical evaluation on a calibrated corridor. On a stochastic microsimulation under a near-unified, approximately budget-matched protocol (TransitDuet 300 episodes per seed; search baselines 320 episodes = 20 lower warmup + 300 search-evaluation), TransitDuet matches or improves on every baseline on its strongest metrics: composite cost 1.418 vs 1.488 for the next-best baseline (CMA-ES, -4.7%) and 1.544 for Fixed (-8.2%). GA in our 300-episode from-scratch setup exhibits a 1-in-3 cold-start failure mode (one of three seeds converges to high CV) that pushes its mean composite to 1.800 with std 0.458, illustrating the unreliability of population-based search at this budget.

Results preview.

We evaluate on a high-frequency bus corridor with 22 stations, 264 daily trips, demand stochasticity $\sigma_{\text{demand}} = 0.15$, and elastic fleet $N_{\text{fleet}} \in [8, 16]$ sampled per episode. Under a near-unified, approximately budget-matched protocol (300 vs 320 simulator episodes per seed for TransitDuet vs the search baselines), TransitDuet improves on the strongest external baseline (CMA-ES) by 2.5% on wait, 12.1% on CV, and 4.7% on composite cost; the overshoot margin over CMA-ES (1.60 vs 1.63) is within seed noise and we do not claim a substantive overshoot advantage. Against the Fixed timetable the gap widens to 7–11% on every metric. GA in this protocol shows a 1-in-3 cold-start failure mode that brings its composite to 1.800 ± 0.458 . TransitDuet degrades only 27% on wait when route stochasticity is doubled beyond the training distribution.

2 Related work

2.1 Reinforcement learning for bus holding control

The application of reinforcement learning to real-time bus holding control has attracted considerable attention in recent years, driven by the promise of adaptive policies that can respond to stochastic

passenger demand and traffic conditions. Alesiani et al. [11] formulated holding for high-frequency services as a value-based RL problem, showing that a Q-learning controller could reduce headway dispersion compared to schedule-based rules. Wang et al. [2] subsequently studied multi-agent deep RL for the bus-bunching problem, casting each bus as an independent agent with a shared policy network and demonstrating that learned holding actions stabilize headways across an entire corridor without explicit communication. More recently, Zhang et al. [12] proposed RE-SAC, an ensemble-critic variant of soft actor-critic for bus fleet control that disentangles aleatoric and epistemic risks via the ensemble disagreement signal and a lower-confidence-bound policy update; we adopt this critic structure at both levels of TransitDuet. The shared-parameter paradigm common to these works scales gracefully with fleet size and implicitly encourages cooperative behavior through shared gradient updates.

Despite these advances, existing RL-based holding controllers share a fundamental limitation: they operate under a fixed, externally supplied timetable. The planned departure times and target headways are treated as immutable inputs, meaning that the holding policy has no mechanism to communicate execution-level feedback—such as systematic schedule infeasibility or recurrent bunching patterns—back to the planning stage. When the timetable itself is poorly calibrated to actual operating conditions, even an optimal holding policy can only mitigate, not resolve, the resulting service irregularity. TransitDuet addresses this gap by embedding the holding controller as the low-level agent within a bi-level framework, where the upper policy steers the lower’s headway target each dispatch via a goal-conditioned coupling.

2.2 Bus timetable optimization

Timetable design has traditionally been approached through mathematical programming and meta-heuristic search. Ibarra-Rojas et al. [13] formulated the problem as a mixed-integer program that jointly determines departure times and vehicle assignments to minimize a weighted combination of passenger waiting time and operator cost, solving it via branch-and-bound with valid inequalities. Gkiotsalitis et al. [1] proposed a bi-level model in which the upper level selects headways to minimize total passenger cost while the lower level solves a vehicle scheduling problem, solved using an adaptive large neighborhood search. Frequency-setting variants, such as the work of Cats et al. [14], optimize the number of departures per period rather than individual departure times, and typically rely on assignment models to estimate passenger flows under each candidate frequency vector.

A common thread in this literature is the reliance on static demand forecasts and deterministic travel-time assumptions. Timetables are optimized offline against expected conditions and then handed off to operations without a feedback loop. When actual conditions deviate—due to traffic incidents, demand surges, or driver variability—the timetable may become infeasible, and holding controllers are left to absorb the mismatch. Moreover, the combinatorial structure of these formulations limits scalability: solution times grow rapidly with the number of trips and stops, making re-optimization during the operating day impractical. TransitDuet replaces this open-loop paradigm with a closed-loop one. The high-level agent learns an adaptive target-headway policy that maps aggregate state features to per-dispatch target-headway shifts on top of a fixed nominal timetable, and it receives gradient signals from low-level execution via the cross-level coupling described in Section 4.4, enabling the planner to adapt its decisions based on how well the lower-level controller realises the requested headway under the current demand and travel-time conditions.

2.3 Hierarchical and multi-timescale reinforcement learning

Hierarchical reinforcement learning decomposes complex tasks into temporally extended sub-problems. The options framework of Sutton et al. [3] introduced the notion of temporally abstract actions, each consisting of an initiation set, an intra-option policy, and a termination condition, enabling planning over multiple timescales within a single MDP. Feudal networks [15] operationalize this idea with a manager that sets sub-goals in a learned latent space and a worker that executes primitive actions to reach them, training both levels end-to-end with the manager’s policy gradient modulated by the worker’s goal-reaching success. HIRO [4] further improved sample efficiency by introducing off-

policy corrections that relabel high-level goals in hindsight, making hierarchical training practical in continuous-control domains. Beyond goal-conditioned hierarchies, Lee et al. [16] and Metelli et al. [17] studied the multi-timescale setting more broadly, analyzing the interaction between agents operating at different decision frequencies and establishing regret bounds for the resulting non-stationary sub-problems.

While these methods provide powerful abstractions for temporal hierarchy, most assume either synchronous execution—where the high-level decision triggers immediately before a fixed-length sequence of low-level steps—or a shared state space across levels. In transit operations, the planning and control timescales are fundamentally asynchronous: the timetable agent acts once per planning horizon (e.g., every 15–30 minutes), while the holding agent acts at every stop visit (every 1–3 minutes), and the two levels observe different state representations at non-aligned decision epochs. TransitDuet handles this asynchrony with a goal-conditioned coupling in the spirit of HIRO [4] and SHIRO [18] (Section 4.4): the upper outputs a per-dispatch target-headway shift, and the lower’s Lagrangian cost is computed against the resulting goal headway, with no upper-advantage injection into the lower’s transitions. We tested two architectural alternatives end-to-end — a literal cross-advantage augmentation following HAAR [9] with a PIPER-style reachability gate [10], and a three-channel coupling that uses δ_t as a launch-time perturbation with hindsight reward credit — and found the goal-conditioned design more reliable in our setting (Section 5.2).

2.4 Constrained reinforcement learning

Many practical RL deployments must satisfy constraints beyond reward maximization. Lagrangian relaxation is the most widely used approach: Tessler et al. [6] proposed reward-constrained policy optimization, which augments the policy gradient with dual variables corresponding to cost constraints and performs primal-dual updates to steer the policy toward feasibility. Stooke et al. [19] improved upon this with PID-based Lagrange multiplier updates that reduce oscillation and speed convergence in the constrained setting. These methods are effective for soft constraints, where occasional small violations are tolerable, but they offer no worst-case guarantees—the Lagrange multiplier may fail to grow fast enough to prevent persistent violations during training.

For settings that demand hard constraint satisfaction, Miryoosefi et al. [8] introduced ApproPO, which formulates the constrained problem as an online convex optimization over the space of occupancy measures and applies a measurement-based projection oracle to enforce feasibility at every iterate. The θ -OGD variant provides a practical implementation that projects policy parameters onto the feasible set using online gradient descent with approximate projections. In the multi-agent domain, Pan et al. [5] proposed RoboDuet for humanoid locomotion, where two coupled agents—an upper-body controller and a lower-body locomotor—exchange cross-advantage terms that capture how each agent’s action quality is perceived under the other’s value function, enabling tight coordination without centralized control.

TransitDuet combines elements from both lines of work to address the constraint structure of transit operations. Fleet-size feasibility is handled as a soft constraint via the θ -OGD measurement-projection scheme of Miryoosefi et al. [8], which adaptively reweights an overshoot penalty in the upper-level reward; in expectation this drives the policy toward feasible fleet utilisation, but per-episode fluctuations under elastic fleet sampling and demand noise can still produce small, bounded overshoot. Headway uniformity is similarly handled as a soft constraint via a Lagrangian penalty with adaptive multiplier updates in the low-level holding policy. The cross-level coordination itself is managed by the goal-conditioned coupling described in Section 4.4, which extends the synchronous goal-setting of HIRO to the asynchronous, multi-timescale regime that characterises the planning–control interface.

3 Problem formulation

We formalise joint adaptive target-headway planning and holding control as a bi-level Markov decision process (MDP) operating over a single bus route with time-varying passenger demand. Concretely, the upper level is responsible for setting per-dispatch target headways (a discrete-time,

low-frequency decision atop a fixed nominal timetable) and the lower level enacts those targets through real-time holding decisions; we use “planning” in the closed-loop adaptive sense throughout, not in the offline mixed-integer-programming sense common in the bus-timetable optimisation literature. This section first describes the transit system model (Section 3.1), then defines the upper- and lower-level MDPs (Section 3.2), states the constraint specification (Section 3.3), and concludes with a discussion of the timescale separation that motivates our algorithmic design (Section 3.4).

3.1 Bus transit system model

We consider a single bidirectional bus route consisting of M stations indexed $\{1, \dots, M\}$. Service is provided according to a timetable that dispatches a total of 264 trips per day, alternating between the two directions. Passenger demand is drawn from time-varying origin–destination (OD) matrices that capture morning, midday, and evening peaks. At any point in time, at most $N_{\text{fleet}}^{\max} = 25$ buses may be concurrently in service; each trip is characterized by a launch time and a travel direction. The per-dispatch target headway—the desired time gap between consecutive launches in the same direction, communicated to the lower-level controller as a goal—is the primary planning lever, while per-station holding times constitute the real-time control lever.

3.2 Bi-level MDP formulation

We decompose the joint optimization into two coupled MDPs that share a common environment but differ in observation granularity, action semantics, and decision frequency.

3.2.1 Upper-level MDP (adaptive target-headway planning)

The upper-level policy π_U sets a per-dispatch target headway atop the fixed nominal timetable. It is invoked once per dispatch event ($M \approx 264$ events per episode, ≈ 132 per direction; one episode is one full service day), so its sampling cadence is the realised headway itself rather than a fixed wall-clock timer.

State.

The upper state encodes both system-level summary statistics and the holding-feedback (HoldFB) block that closes the lower-to-upper loop (Section 4.4):

$$s_U = \left[\underbrace{\frac{\text{hour}}{24}, \frac{\text{demand}}{1000}, \frac{\text{fleet_on_route}}{N_{\text{fleet}}^{\max}}, \frac{\text{prev_headway}}{600}}_{\text{system summary, dims 0–4}}, \underbrace{\frac{\bar{h}_{\text{hold}}^{\text{up}}}{60}, \frac{\sigma_{\text{hold}}^{\text{up}}}{60}, \frac{\bar{h}_{\text{hold}}^{\text{down}}}{60}}_{\text{HoldFB, dims 5–7}}, \underbrace{\frac{h^{\text{target}}}{600}, \text{dir}, \frac{N_{\text{fleet}}}{20}}_{\text{trip context, dims 8–10}} \right] \in \mathbb{R}^{11}. \quad (1)$$

where each component is normalized to facilitate learning. Dims 5–7 are the rolling holding statistics over the last 10 completed trips (see Section 4.4); the –HoldFB ablation in Section 5.3 zeroes these three dimensions. Dim 10 is the per-episode elastic fleet budget $N_{\text{fleet}} \in [8, 16]$, which the policy must condition on to produce the Pareto frontier of Section 5.4.

Action.

At each dispatch event the upper policy outputs a scalar headway shift that adjusts the lower controller’s target headway for the dispatched trip:

$$a_U = \delta_t \in [-120, 120] \text{ s}. \quad (2)$$

The bus’s scheduled launch time stays at the baseline value t_{base} (from a timetable with $H = 360$ s); the target headway h^{target} communicated to the lower level for that trip is reset to $h^{\text{target}} + \delta_t$. The

lower controller then realises the operational schedule via holding (Section 3.2.2): a positive δ_t asks the lower to track a longer headway, which it implements by holding the trailing bus at intermediate stations; a negative δ_t asks for tighter regulation. This casts the planning problem as adaptive headway-target setting, with the lower as the realisation layer — equivalent in expressive power to a per-trip headway choice while preserving the lower’s training distribution under upper exploration (Section 4.4). It also matches the goal-conditioned hierarchical RL formulation of HIRO [4].

Reward.

The upper reward is computed at the episode level from a measurement vector (defined in Section 3.3). Because each upper action influences the system over an extended horizon through the downstream holding decisions it induces, assigning credit to individual dispatch events is nontrivial; we defer the reward construction to Section 4.

3.2.2 Lower-level MDP (holding control)

The lower-level policy π_L decides how long each bus should dwell beyond its minimum stop time at every station. A decision is triggered at each station arrival for each active bus, producing a high volume of decision epochs throughout the day.

State.

The lower state captures the local context of an individual bus:

$$s_L = [\text{bus_id}, \text{station_id}, \text{hour}, \text{direction}, h_{\text{fwd}}, h_{\text{bwd}}, \hat{d}, \Delta h, v_1, \dots, v_K] \in \mathbb{R}^{8+K}, \quad (3)$$

where h_{fwd} and h_{bwd} denote the forward and backward headways, $\hat{d}$ is the estimated dwell time, Δh is the deviation of the current headway from h^{target} , and $v_1, \dots, v_K$ are recent inter-station speeds that encode local traffic conditions.

Action.

The lower action is a scalar holding time:

$$a_L \in [0, 60] \text{ seconds}. \quad (4)$$

Reward and cost.

The immediate lower reward is a function of headway deviation from the target set by the upper level. We define the per-step cost as

$$c(s_L, a_L) = \min\left(\frac{(h_{\text{fwd}} - h^{\text{target}})^2}{(h^{\text{target}})^2}, 1\right), \quad (5)$$

which penalizes headway irregularity in a scale-invariant manner and is clipped at 1 for numerical stability.

3.3 Constraint specification

The bi-level MDP is subject to two soft constraints, both evaluated through a measurement vector collected at the end of each episode.

Measurement vector.

At the conclusion of a service day, we compute:

$$\mathbf{z} = [\text{avg_wait_min}, \text{peak_fleet}, \text{bunching_rate}]. \quad (6)$$

Soft constraint (fleet budget).

The peak number of buses simultaneously on route should respect the per-episode fleet budget $N_{\text{fleet}} \in [N_{\text{fleet}}^{\min}, N_{\text{fleet}}^{\max}]$ sampled at episode start. We enforce this softly via an overshoot² penalty in the upper reward:

$$\text{overshoot}(\mathbf{z}, N_{\text{fleet}}) = \max(0, \text{peak_fleet} - N_{\text{fleet}})^2, \quad (7)$$

weighted by an adaptive coefficient maintained by θ -OGD (see Section 4.5). To prevent unbounded violation under noisy demand, the environment imposes an outer ceiling $N_{\text{fleet}}^{\max} + 3$ buses, which bounds but does not zero out the realised overshoot.

Soft constraint (headway uniformity).

The expected per-step cost must remain below a threshold:

$$\mathbb{E}_{\pi_U, \pi_L} [c(s_L, a_L)] \leq c_{\text{lim}}, \quad (8)$$

which encourages even spacing of buses along the route. Satisfaction of this constraint is enforced via Lagrangian relaxation within the lower-level policy (see Section 4).

3.4 Timescale separation

A defining characteristic of the bi-level formulation above is the pronounced separation of timescales between the two decision layers. The upper policy π_U acts at dispatch events, which occur roughly every five minutes on average. The lower policy π_L , by contrast, acts at every station arrival for every active bus—a frequency on the order of once every 30 seconds system-wide when multiple buses are in service. Consequently, hundreds of lower-level decisions may intervene between two consecutive upper-level decisions, and the two layers never act simultaneously.

This asynchrony distinguishes our setting from the semi-Markov options framework [3] and feudal reinforcement learning [20, 15], which assume that (i) the manager and worker share a common state space or a fixed abstraction thereof, and (ii) the manager selects a subgoal that the worker executes to completion before the manager acts again. Neither assumption holds here: the upper state $s_U \in \mathbb{R}^{11}$ summarizes fleet-level statistics plus the HoldFB block whereas the lower state $s_L \in \mathbb{R}^{8+K}$ encodes bus-level kinematics, and both policies issue decisions driven by environment events rather than by a synchronous clock or a predefined option duration.

More concretely, a single upper action (a target-headway shift δ_t) remains in effect for the next dispatch while many buses traverse multiple stations under independent lower-level control; the upper policy cannot pause and wait for all induced lower decisions to conclude. Standard hierarchical RL methods that propagate value or advantage across levels therefore require modification to handle the fully asynchronous, event-driven interaction between π_U and π_L . We address this challenge through a goal-conditioned cross-level coupling, detailed in Section 4.

4 TransitDuet framework

This section presents the TransitDuet architecture and its constituent algorithms. Figure 1 gives an overview of the bi-level system. Section 4.1 motivates the decoupled dual-network design. Section 4.2 and Section 4.3 detail the upper-level timetable policy and lower-level holding control policy, respectively. Section 4.4 describes the goal-conditioned cross-level coupling that links the two levels across timescales. Section 4.5 describes the measurement-driven scheme that adaptively penalises fleet-overshoot at the upper level. Section 4.6 assembles the full training procedure. Section 4.8 introduces the target-policy-corrected lower SAC (TPC) training fix that closes the gap to a hand-tuned fixed timetable.

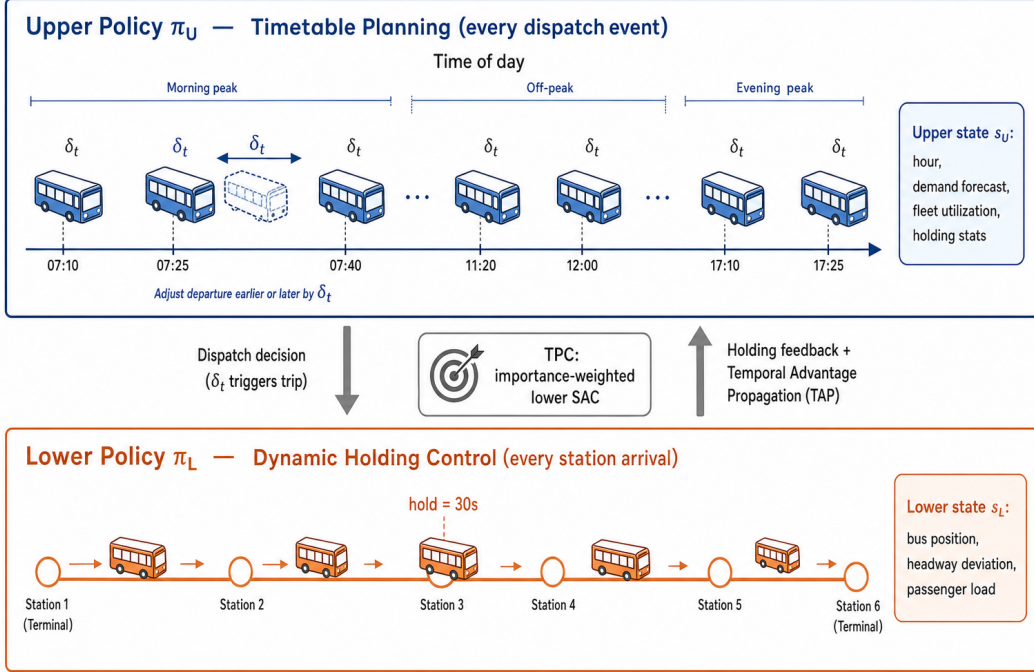


Fig. 1. TransitDuet bi-level architecture. The upper policy π_U acts at each dispatch event (timescale: minutes), choosing a per-dispatch goal shift δ_t that adjusts the lower’s target headway from h^{target} to $h^{\text{target}} + \delta_t$. The lower policy π_L acts at each station arrival (timescale: seconds), choosing a holding duration that drives the realised headway toward this goal. The coupling is goal-conditioned (HIRO-style): no upper signal is injected into the lower’s transitions or reward beyond the shifted target. TPC importance-weighted training of the lower SAC corrects for any residual covariate shift induced by changing target headways during upper exploration.

4.1 Architecture overview

TransitDuet comprises two independent policy networks operating at different timescales and over different state-action spaces. The upper policy π_U is a lightweight Gaussian MLP that selects a per-dispatch headway shift at each dispatch event. The lower policy π_L is a RE-SAC–Lagrangian controller that selects holding durations at every station arrival, with parameters shared across all buses in the fleet.

A key design choice is that the two networks share no encoder parameters. This stands in contrast to the shared-body, dual-head architecture of RoboDuet [5], which couples an upper style policy and a lower tracking policy through a common proprioceptive encoder. Shared encoders are appropriate when the two levels observe the same state at each time step—as in synchronous robot locomotion—but are ill-suited here for two reasons. First, s_U and s_L live in different spaces: s_U encodes route-level demand forecasts and fleet status, whereas s_L encodes per-bus headway deviation, passenger load, and local occupancy. Second, the two levels are triggered asynchronously. The upper policy acts once per dispatch event (roughly once per h^{target} seconds), while the lower policy acts at every station arrival of every active bus, producing hundreds of decisions per upper-level step. Forcing these heterogeneous signals through a shared trunk would create conflicting gradient directions.

The coupling between the two levels is instead a goal-conditioned action channel: the upper sets the lower’s headway target each dispatch, as described in section 4.4. This clean separation lets us train both networks with off-policy soft actor-critic: a RE-SAC ensemble Q upper that reuses each per-dispatch transition many times via a replay buffer, and the same RE-SAC ensemble structure augmented with a Lagrangian dual variable at the lower level for headway-constraint satisfaction.

4.2 Upper-level timetable policy

The upper policy π_U maps the route-level state s_U to a scalar headway-shift $a_U = \delta_t \in [-120, 120]$ s. The bus’s scheduled launch time stays at the baseline value (t_{base} from an $H = 360$ s schedule), and the target headway communicated to the lower controller is reset for that trip to $h^{\text{target}} + \delta_t$. The lower’s Lagrangian cost (Section 4.3) is then computed against this dispatch-specific target, so δ_t acts as a goal in the sense of HIRO [4].

Network architecture.

The policy is parameterized as a two-layer MLP with 64 hidden units per layer and ReLU activations:

$$\mu_\phi(s_U) = \sigma(W_3 \text{ReLU}(W_2 \text{ReLU}(W_1 s_U + b_1) + b_2) + b_3), \quad (9)$$

where $\sigma(\cdot)$ denotes the element-wise sigmoid function and ϕ collects all weights and biases. The sigmoid output is affinely mapped to the action range:

$$a_U = -120 + 240 \mu_\phi(s_U) \in [-120, 120] \text{ s}. \quad (10)$$

Exploration is achieved by additive Gaussian noise with a learnable log-standard-deviation $\log \sigma_\phi$ shared across states:

$$a_U \sim \mathcal{N}(\mu_\phi(s_U), \sigma_\phi^2). \quad (11)$$

Off-policy SAC training (RE-SAC upper).

The upper policy is trained as an off-policy soft actor-critic with an ensemble of $K=10$ Q-networks and a lower-confidence-bound (LCB) value used in the policy improvement step — the same RE-SAC-style structure [12] used at the lower level (section 4.3), without the Lagrangian. Each dispatch event i contributes a transition $(s_{U,i}, a_{U,i}, R_U^{(i)}, s_{U,i+1}, d_i)$ to a 50k-capacity replay buffer; the per-dispatch reward $R_U^{(i)}$ is the hindsight-credited reward of Equation (23) (section 4.4). The ensemble Q-networks are trained against a single shared mean-target to prevent ensemble divergence,

$$y_i = R_U^{(i)} + \gamma (1 - d_i) \left(\frac{1}{K} \sum_{k=1}^K \bar{Q}_k(s_{U,i+1}, \tilde{a}) - \alpha_U \log \pi_U(\tilde{a} \mid s_{U,i+1}) \right), \quad (12)$$

where $\bar{Q}_k$ is the k -th target network, $\tilde{a} \sim \pi_U(\cdot \mid s_{U,i+1})$, and the target is clamped to $[-50, 50]$; the per-Q MSE is augmented with a small ensemble-spread penalty $\beta_{\text{ood}} \cdot \text{std}_k(Q_k)$ that encourages the ensemble to disagree on out-of-distribution actions. The policy is improved by maximising the LCB value:

$$\mathcal{L}_{\pi_U}(\phi) = \mathbb{E}_{s \sim \mathcal{D}_U, a \sim \pi_U} \left[\alpha_U \log \pi_U(a \mid s) - (\bar{Q}(s, a) + \beta_{\text{LCB}} \sigma_Q(s, a)) \right], \quad (13)$$

where $\beta_{\text{LCB}} = -2$, so the LCB value $\bar{Q} + \beta_{\text{LCB}} \sigma_Q$ subtracts twice the ensemble disagreement from the mean. Auto-entropy adjusts α_U toward the target entropy $-\dim(a_U) = -1$, capped at $\alpha_{\text{max}}^U = 0.05$ in the goal-conditioned configuration (the upper action is a 1-dimensional headway shift, so the entropy target $-\dim(a_U) = -1$ saturates quickly without a low cap). The choice of an off-policy SAC upper rather than an on-policy estimator is deliberate: dispatches are scarce ($M \approx 264$ per episode), so a replay buffer reuses each transition many times across gradient updates.

4.3 Lower-level holding control policy

The lower policy π_L selects a signed holding adjustment $a_L \in [-a_{\text{max}}, a_{\text{max}}]$ at each station arrival, where $a_{\text{max}} = 60$ s. A positive a_L holds the bus at the station for a_L extra seconds beyond the minimum dwell, while a negative a_L shortens the next inter-station idle by $|a_L|$ seconds (capped by the realised idle slack); the simulator clips the realised holding to $[0, a_{\text{max}}]$ so a release action that would advance the bus past its physical readiness has no effect. It is trained with a distributional soft actor-critic augmented by a Lagrangian cost term for headway-deviation constraint enforcement [7]. All buses share the same policy parameters, enabling fleet-wide generalization.

Gaussian policy with tanh squashing.

A two-layer MLP with 64 hidden units per layer and ReLU activations produces a mean $\mu_\psi(s_L)$ and log-standard-deviation $\log \sigma_\psi(s_L)$. The raw sample $u \sim \mathcal{N}(\mu_\psi, \sigma_\psi^2)$ is squashed through a tanh transform and rescaled:

$$a_L = a_{\max} \tanh(u). \quad (14)$$

The log-probability accounts for the change of variables:

$$\begin{aligned} \log \pi_L(a_L | s_L) &= \log \mathcal{N}(u; \mu_\psi, \sigma_\psi^2) \\ &\quad - \log(1 - \tanh^2(u)) - \log a_{\max}. \end{aligned} \quad (15)$$

Reward Q-ensemble (RE-SAC, $K=10$).

Rather than the standard twin-Q double-clipped critic of SAC, the lower uses the RE-SAC critic of Zhang et al. [12], which replaces the double-clip with an ensemble of $K=10$ independently-initialised Q-networks $\{Q_{\omega_k}\}_{k=1}^K$, each a two-layer MLP with 64 hidden units, and uses a lower-confidence bound rather than the min-of-twin for policy improvement. To prevent ensemble divergence, all members are trained against a single shared mean target,

$$\begin{aligned} y &= r + \gamma \left(\frac{1}{K} \sum_{k=1}^K Q_{\bar{\omega}_k}(s'_L, a'_L) \right. \\ &\quad \left. - \alpha \log \pi_L(a'_L | s'_L) \right), \end{aligned} \quad (16)$$

with $a'_L \sim \pi_L(\cdot | s'_L)$, and the per- Q_k loss is augmented with a small ensemble-spread regulariser,

$$\begin{aligned} \mathcal{L}_Q(\omega_k) &= \mathbb{E}_{(s_L, a_L, r, s'_L) \sim \mathcal{D}} \left[(Q_{\omega_k}(s_L, a_L) - y)^2 \right] \\ &\quad + \beta_{\text{ood}} \text{std}_k(Q_{\omega_k}(s_L, a_L)), \end{aligned} \quad (17)$$

with $\beta_{\text{ood}} = 0.1$, which encourages members to disagree on out-of-distribution actions. Target parameters $\bar{\omega}_k$ track ω_k via Polyak averaging with $\tau = 0.005$. Policy improvement is performed against the ensemble lower-confidence bound (LCB),

$$\bar{Q}(s, a) = \frac{1}{K} \sum_k Q_{\omega_k}(s, a), \quad \sigma_Q(s, a) = \text{std}_k(Q_{\omega_k}(s, a)). \quad (18)$$

A standard SAC twin-Q critic corresponds to the special case $K=2$ without the LCB; we adopt the $K=10$ ensemble to obtain calibrated uncertainty for the lower’s actor loss below.

Cost Q-network and Lagrangian relaxation.

A separate single-MLP cost critic $Q_\xi^c(s_L, a_L)$ (same hidden width 64) estimates the expected cumulative headway-deviation cost. The lower-level actor loss balances entropy, the ensemble LCB value, and the constraint:

$$\begin{aligned} \mathcal{L}_{\pi_L}(\psi) &= \mathbb{E}_{s_L \sim \mathcal{D}, a_L \sim \pi_L} \left[\alpha \log \pi_L(a_L | s_L) \right. \\ &\quad \left. - (\bar{Q}(s_L, a_L) + \beta_{\text{LCB}} \sigma_Q(s_L, a_L)) \right. \\ &\quad \left. + \lambda Q_\xi^c(s_L, a_L) \right], \end{aligned} \quad (19)$$

with $\beta_{\text{LCB}} = -2$ (the policy is improved against the lower confidence bound) and $\lambda \geq 0$ the Lagrange multiplier. The dual is updated to satisfy the cost-limit constraint c_{lim} :

$$\begin{aligned} \lambda &\leftarrow \exp \left[\text{clamp}(\log \lambda + \eta_\lambda (\mathbb{E}[Q_\xi^c] - c_{\text{lim}}), \right. \\ &\quad \left. -10, 5) \right], \end{aligned} \quad (20)$$

with $\eta_\lambda = 10^{-4}$. Updating in log-space ensures $\lambda > 0$ and the clamp prevents numerical instability.

Auto-entropy tuning.

The temperature α is adjusted to maintain a target entropy $\bar{\mathcal{H}} = -\dim(a_L) = -1$:

$$\alpha \leftarrow \min\left(\alpha_{\max}^L, \exp(\log \alpha - \eta_\alpha (\log \pi_L(a_L | s_L) + \bar{\mathcal{H}}))\right), \quad (21)$$

with $\alpha_{\max}^L = 0.1$ to prevent entropy collapse and to let the goal-conditioned coupling settle as the policy gains confidence.

4.4 Cross-level coupling: goal-conditioned design

Following the goal-conditioned hierarchical RL design of HIRO [4] and SHIRO [18], TransitDuet couples the two policies through a single goal channel on the action side, complemented by two auxiliary signals (state and reward) used only in the upper direction. The upper outputs a target-headway shift; the lower’s Lagrangian cost is then a function of how closely the realised headway tracks the resulting goal headway. There is no upper-advantage injection into the lower reward; the lower remains a constraint-aware controller whose constraint target is set by the upper. This turned out to be more reliable than two architectural alternatives we evaluated end-to-end (Section 5.2): a direct cross-advantage augmentation in the spirit of HAAR [9] with a PIPER-style reachability gate [10], and a three-channel coupling that uses δ_t as a launch-time perturbation with hindsight reward credit instead of a goal shift.

Goal channel: target-headway shift.

At each dispatch event, the upper outputs $\delta_t \in [-120, 120]$ s. In contrast to a direct timetable perturbation that would shift the bus’s effective launch time, δ_t instead reinterprets the lower’s target headway from h^{target} (the baseline 360 s value) to $h^{\text{target}} + \delta_t$ for the duration of that trip. The launch schedule itself is left untouched, while the lower’s Lagrangian constraint becomes

$$\mathbb{E}_{\tau \sim \pi_L} [|H_{\text{real}}(\tau) - (h^{\text{target}} + \delta_t)|] \leq c_{\text{high}}, \quad (22)$$

where $H_{\text{real}}(\tau)$ is the realised inter-bus headway over the trip lifecycle τ . The upper policy thus learns to steer the lower’s constraint target rather than to inject noise into its observations. By construction, an active upper policy can only adjust the goal that an already-trained lower controller is asked to satisfy; this preserves the lower’s training distribution and avoids the covariate shift that destabilises three-channel coupling under heavy upper exploration.

State channel: holding feedback (HoldFB).

The upper state s_U contains a 3-dimensional holding-feedback block at indices 5–7 (Eq. 1; Section 4.7) carrying a rolling 10-trip summary of the lower policy’s recent holding decisions: $[\bar{h}_{\text{hold}}^{\text{up}}, \sigma_{\text{hold}}^{\text{up}}, \bar{h}_{\text{hold}}^{\text{down}}]$ normalised to the action range. Through this channel, the upper policy observes how aggressively the lower controller is intervening to satisfy the goal and adapts δ_t to avoid setting goals that systematically force the lower into prolonged over-holding.

Reward channel: trip-lifecycle hindsight credit.

At episode end, we compute the realised inter-dispatch gap g_i for each dispatch i from the actual launch times of each bus, and form a per-dispatch deviation $\Delta_i = |g_i - \bar{g}|/\sigma_g$ where $(\bar{g}, \sigma_g)$ are the per-direction mean and standard deviation. The upper reward at dispatch i is then composed of the episode-level system reward and a per-dispatch hindsight credit:

$$R_U^{(i)} = R_{\text{sys}}(\mathbf{z}) - \alpha_H \cdot \frac{\Delta_i - \bar{\Delta}}{\sigma_{\Delta}}, \quad \alpha_H = 0.5, \quad (23)$$

where R_{sys} is the belief-weighted multi-objective reward (Eq. 28) and the second term recentres the deviation across the episode so that goals whose realised lifecycles produced more regular gaps than the episode average earn positive credit. This is upper-only credit assignment; no signal flows back into the lower transitions.

Lower-level reward.

The lower’s reward is unchanged from Section 4.3: the standard environment reward augmented by the headway-deviation cost handled by the Lagrangian. The only effect of upper coupling on the lower is via Eq. 22, which redirects the constraint target each dispatch. We deliberately do not inject upper-level reward signals into lower transitions; we tested a literal HAAR-style reverse cross-advantage augmentation (with and without a PIPER reachability gate) and found that even with a clipped advantage and the gate engaged, mean composite cost was 4% worse and seed variance $1.7\times$ higher than the goal-conditioned coupling reported here.

Training-stage schedule.

Cross-level coupling is gated by a simple two-stage schedule. During the first $e_{\text{warm}} = 30$ episodes the upper policy is inactive ($\delta_t = 0$ so the lower trains against the baseline target h^{target}); from episode e_{warm} onward the upper activates and the goal channel engages. The state and reward channels operate only on the upper side and so are inactive during warmup by construction.

4.5 Measurement-driven adaptive penalty for fleet utilisation

The upper policy must respect operational targets on fleet size, average passenger waiting time, and bus bunching rate. Rather than adding fixed penalty terms to the reward (which requires hand-tuning a coefficient per metric), we use an adaptive scheme inspired by the ApproPO framework [8]: a measurement-driven online gradient descent (θ -OGD) on the reward-weight vector, which adapts the relative penalty strengths to the realised episode statistics. Unlike full ApproPO, our scheme does not solve a nested best-response oracle and therefore does not provide ApproPO’s feasibility guarantees; in practice it reweights the reward toward the directions where the recent measurement is furthest from its target, and we observe in section 5 that fleet overshoot stabilises at a small bounded value but is not driven to zero.

Measurement vector.

At the end of each episode, we compute a three-dimensional measurement vector:

$$\mathbf{z} = [\bar{w}, n_{\text{peak}}, b_{\text{rate}}] \in \mathbb{R}^3, \quad (24)$$

where $\bar{w}$ is the average passenger waiting time, n_{peak} is the peak fleet size (maximum number of buses simultaneously in service), and b_{rate} is the bunching rate (fraction of arrivals with headway below a threshold).

Weight vector and reward modulation.

A weight vector $\boldsymbol{\theta} \in \mathbb{R}^3$, initialized as $\boldsymbol{\theta}_0 = [-0.5, -0.3, -0.2]$, modulates the upper-level reward:

$$\begin{aligned} R_U &= \boldsymbol{\theta}^\top \mathbf{p}, \\ \mathbf{p} &= [\bar{w}, (\max(0, n_{\text{peak}} - N_{\text{fleet}}))^2, b_{\text{rate}}]. \end{aligned} \quad (25)$$

The quadratic penalty on fleet overshoot $(\max(0, n_{\text{peak}} - N_{\text{fleet}}))^2$ ensures that small violations incur proportionally less cost than large ones, providing a smooth gradient.

Online gradient descent update.

The weight vector is updated by online gradient descent (OGD) on a loss that drives each measurement toward its target:

$$\ell_t = [-\bar{w}_t, -\max(0, n_{\text{peak},t} - N_{\text{fleet}}), -b_{\text{rate},t}], \quad (26)$$

$$\boldsymbol{\theta}_{t+1} = \Pi_{\mathcal{B}}(\boldsymbol{\theta}_t + \eta_t \ell_t), \quad \eta_t = \frac{\eta_0}{\sqrt{t}}, \quad (27)$$

where $\Pi_{\mathcal{B}}$ projects onto the unit ball $\{\boldsymbol{\theta} : \|\boldsymbol{\theta}\|_2 \leq 1\}$ and η_0 is the base learning rate. The $1/\sqrt{t}$ schedule is motivated by standard online-convex-optimisation regret analysis (where projected OGD on a convex loss enjoys an $O(\sqrt{T})$ regret bound and a sublinear cumulative-constraint-violation

Algorithm 1 TransitDuet training procedure

Require: Episode budget E , upper warmup $e_{\text{warm}} = 30$, lower update frequency f_L , batch size B , hindsight credit weight α_H

- 1: Initialise upper policy π_U , lower policy π_L (RE-SAC ensemble of $K=10$ reward critics + single cost critic), replay buffer $\mathcal{D}$, Lagrange multiplier λ , entropy temperature α , θ -OGD weights, EMA target upper $\bar{\pi}_U$
- 2: for episode $e = 1, \dots, E$ do
- 3: Reset simulation; sample $N_{\text{fleet}} \sim \text{Uniform}\{8, \dots, 16\}$ if elastic
- 4: $\mathcal{T}_U \leftarrow \emptyset, \mathcal{T}_L \leftarrow \emptyset$
- 5: while episode not done do
- 6: if dispatch event triggered then
- 7: if $e \leq e_{\text{warm}}$ then
- 8: Set $\delta_t = 0$ (baseline timetable; upper inactive)
- 9: else
- 10: Sample δ_t from the TPC behaviour mixture (section 4.8); set the trip’s target headway $\leftarrow h^{\text{target}} + \delta_t$ (launch time unchanged)
- 11: Record $(s_{U,i}, a_{U,i})$ in $\mathcal{T}_U$
- 12: end if
- 13: end if
- 14: if station arrival for any bus b then
- 15: Observe s_L ; sample $a_L \sim \pi_L(\cdot \mid s_L)$; apply holding a_L
- 16: Observe r_L, c_L, s'_L ; store $(s_L, a_L, r_L, c_L, s'_L)$ in $\mathcal{D}$ and $\mathcal{T}_L$
- 17: if step mod $f_L = 0$ **and** $|\mathcal{D}| \geq B$ then
- 18: Sample B transitions; compute TPC IS weights against $\bar{\pi}_U$
- 19: Update π_L critics, policy, λ , α via weighted SAC losses
- 20: Soft-update lower target networks
- 21: end if
- 22: end if
- 23: end while
- 24: Compute episode measurements $\mathbf{z}$, system reward R_{sys} , and per-dispatch hindsight credits via eq. (23)
- 25: Backfill upper transitions in $\mathcal{T}_U$ with $R_U^{(i)}$, push them to the upper replay buffer, and run f_U RE-SAC gradient updates (eq. (12)–(13)) on a sampled mini-batch
- 26: Polyak-update EMA target upper $\bar{\pi}_U$; update θ via OGD on $\mathbf{z}$
- 27: end for

guarantee). We do not claim the same guarantee holds for our learned policy: the inner optimisation here is a non-convex SAC update on a replay buffer rather than the convex best-response oracle that ApproPO requires, and we do not project onto an occupancy-measure feasibility set. We therefore use θ -OGD as a stabilising adaptive penalty whose schedule is principled in its OCO origin but whose feasibility properties on the resulting deep-RL policy are empirical (fig. 5, fig. 6) rather than provable.

The simplification relative to ApproPO is deliberate: because the upper policy is trained off-policy with the θ -modulated reward, repeated SAC gradient updates on the replay buffer act as an approximate best-response step, eliminating the inner optimization loop that ApproPO requires — at the cost of the formal guarantee discussed above.

4.6 Training procedure

Algorithm 1 presents the full TransitDuet training loop, which coordinates the upper-warmup schedule with the upper and lower policy updates, the cross-level coupling channels of section 4.4, and the measurement projection.

Lower-only warmup ($e \leq e_{\text{warm}} = 30$).

The upper policy is forced to $\delta_t = 0$ for every dispatch and the lower policy trains in isolation against the baseline timetable. This warmup is critical: without it, the upper policy would receive auxiliary-channel feedback (HoldFB and hindsight credit) from a random lower policy, which amounts to noise and destabilises upper-level learning. We set $e_{\text{warm}} = 30$ episodes.

Coupled training ($e > e_{\text{warm}}$).

The upper policy activates and the goal channel of section 4.4 engages: each dispatch’s target headway is reset to $h^{\text{target}} + \delta_t$, and the lower’s Lagrangian cost is computed against this dispatch-specific target. The state and reward channels are upper-only and are inactive during warmup by construction. We do not anneal a coupling coefficient; the channel operates at its designed strength from activation onward, with TPC’s IS reweighting (section 4.8) handling any residual covariate shift from changing target headways during upper exploration.

Computational cost.

The lower policy update dominates computation due to the $K=10$ Q-ensemble and cost-critic gradient steps. Each lower update performs $K+3$ backward passes (K ensemble reward critics, one cost critic, one policy, one entropy temperature) on GPU-batched samples from $\mathcal{D}$; the ensemble is implemented as a single batched forward+backward, so wall-clock cost grows sub-linearly with K . The upper policy update is much cheaper: each episode contributes $M \approx 264$ dispatch transitions to a 50k-capacity replay buffer, and we run $f_U = 10$ RE-SAC gradient updates per episode on a 64-sample mini-batch. The TPC IS-weight computation and Polyak target update add $O(M + B)$ scalar/forward-pass operations per episode and are not a bottleneck.

4.7 Auxiliary mechanisms

This subsection collects four secondary mechanisms that interact with the bi-level core. They are isolated as separate ablation rows in Section 5.3.

Holding feedback (HoldFB).

The upper state s_U contains a 3-dimensional holding-feedback block (state indices 5–7) consisting of the lower’s rolling holding statistics $[\bar{h}_{\text{hold}}^{\text{up}}, \sigma_{\text{hold}}^{\text{up}}, \bar{h}_{\text{hold}}^{\text{down}}]$ over a window of the last 10 completed trips, normalised to the action range. This conveys to the upper policy how heavily the lower controller is intervening, allowing δ_t adjustments that reduce systematic over-holding. The $-\text{HoldFB}$ ablation zeros out these three state dimensions.

Belief-weighted MORL (MORL).

The upper reward is a scalarisation of three measurement components:

$$R_U = -\left[w_{\text{wait}} \frac{\text{wait}}{10} + w_{\text{fleet}} \frac{\text{overshoot}^2}{N_{\text{fleet}}} + w_{\text{cv}} \text{CV}\right], \quad (28)$$

where $\mathbf{w} = (w_{\text{wait}}, w_{\text{fleet}}, w_{\text{cv}})$ is computed from $\boldsymbol{\theta}$ -OGD weights modulated by the CS-BAPR belief: under high changepoint probability, mass is shifted from wait/CV to fleet (safety), while during stable windows the converse shift toward wait/CV (quality) is applied. The $-\text{MORL}$ ablation replaces $\mathbf{w}$ with the fixed weights (0.5, 0.25, 0.25).

CS-BAPR belief tracker.

A Bayesian online changepoint detector tracks the run length of the upper reward stream and the standard deviation of the lower’s Q -ensemble. When the changepoint posterior exceeds 0.1 (signalling a regime shift in demand or fleet realisation) the lower’s entropy temperature α is transiently boosted by up to $2\times$ to encourage exploration in the new regime. The $-\text{CS-BAPR}$ ablation disables this boost; the belief weights in MORL also degenerate to the long-run $\boldsymbol{\theta}$ -OGD weights without any short-term modulation.

Hindsight credit assignment.

We summarise the hindsight credit term used by the upper reward; the full derivation is given in section 4.4 (Eq. 23). At episode end we compute the realised inter-dispatch gap g_i per direction,

normalise it to a deviation $\Delta_i = |g_i - \bar{g}|/\sigma_g$, and recentre across the episode so the average dispatch gets zero credit. The $-$ Hindsight ablation drops this term and uses only the episode-level reward, weakening per-dispatch credit assignment.

Elastic fleet (Elastic) and demand stochasticity (DemandNoise).

Two environment-side mechanisms further shape training: $N_{\text{fleet}} \sim \text{Uniform}\{8, 9, \dots, 16\}$ is sampled per episode (Elastic), and the per-hour demand multiplier is sampled from $\mathcal{N}(1, 0.15^2)$ clipped to $[0.3, 2.0]$ (DemandNoise). The $-$ Elastic and $-$ DemandNoise ablations fix $N_{\text{fleet}} = 12$ and remove the demand multiplier, respectively, reducing TransitDuet to a deterministic-environment specialist.

4.8 Target-policy-corrected lower SAC (TPC)

Joint bi-level RL exposes a previously overlooked failure mode that, in our setting, lets a hand-tuned constant-headway timetable beat the bi-level controller on every mean metric: the lower policy is trained alongside an exploring upper policy, so the lower’s training distribution is biased by per-dispatch offsets that do not appear at deployment. Even when the upper learns to output $\delta_t \approx 0$ (effectively recovering the Fixed schedule), the lower’s holding habits no longer match what a Fixed-schedule deployment requires. We address this directly via importance reweighting on the lower’s SAC objective.

Mixed behaviour policy.

Once the upper policy activates ($e > e_{\text{warm}}$), each dispatch samples δ_t from a mixture $\mu = \epsilon \pi_U(\cdot | s_U) + (1 - \epsilon) \mathcal{N}(\bar{\mu}_U(s_U), \sigma_{\text{tgt}}^2)$, with $\epsilon = 0.25$ exploratory probability and target Gaussian std $\sigma_{\text{tgt}} = 20$ s centred at the deterministic mean $\bar{\mu}_U$ of an exponential moving average of the upper policy ($\tau = 0.005$ Polyak rate). The behaviour log-prob $\log \mu(\delta_t | s_U)$ and the dispatch metadata $(s_U, \delta_t, \log \mu)$ are stored alongside each lower transition for later use.

Importance-weighted lower SAC.

For every sampled lower transition, the per-sample importance weight

$$w_t = \min\left(\frac{\bar{\pi}_U(\delta_t | s_{U,t})}{\mu(\delta_t | s_{U,t})}, w_{\max}\right)$$

($w_{\max} = 5$) corrects each loss contribution toward the EMA "deployment" upper distribution. The weighted Q, policy, cost-critic, and Lagrangian losses become $L_Q = \mathbb{E}_t[w_t(Q(s_t, a_t) - y_t)^2]$, $L_\pi = \mathbb{E}_t[w_t(\alpha \log \pi_L(a_t | s_t) - Q^{\text{LCB}}(s_t, a_t) + \lambda Q^c(s_t, a_t))]$, and analogously for the cost critic and Lagrangian update. We monitor effective sample size $\text{ESS}/B = (\sum_t w_t)^2 / (B \sum_t w_t^2)$ to detect weight collapse; in our runs it remains above 0.4 throughout training.

Validation-based checkpoint selection.

The EMA target tracks the current upper with $\tau = 0.005$, which over hundreds of episodes drifts toward the upper’s exploratory mode and gradually weakens the IS correction. We mitigate this with explicit checkpoint selection: every 50 episodes we save (π_U, π_L) to disk; after training we evaluate every checkpoint on 60 fresh validation episodes (independent of the training data) and report the deployment policy as the per-seed checkpoint with the lowest composite cost. Across our three seeds the selected checkpoints are episode 99, 49, and 249 respectively, consistent with the expected drift pattern.

Convergence considerations.

The lower-only warmup ensures that the coupling channels engage only after the lower has reached a reasonable holding policy under the baseline timetable. We initially explored a continuous coupling-coefficient ramp (a β schedule that grows from 0 to a target value over a hundred episodes) but found that the simpler binary-activation schedule combined with TPC’s IS reweighting was both more reliable and easier to tune. We did not observe sensitivity to the precise values of e_1 and e_2 within a factor of two of the defaults, but setting e_1 too small (before the lower policy converges to a stable holding strategy) consistently degraded final performance, as confirmed by the ablation study in section 5.

5 Experiments

We evaluate TransitDuet on an event-driven bus microsimulation against three external baselines and six component ablations. All results are from three independent random seeds.

5.1 Experimental setup

Simulation environment.

We implement an event-driven bus simulation modeling a single bidirectional route with 22 stations (11 per direction), serving 264 scheduled trips per day. Passenger demand follows origin–destination (OD) matrices with time-varying arrival rates calibrated to reproduce morning and evening peaks. The simulation advances at a resolution of $\Delta t = 1$ s. Passenger arrivals and alighting events are updated every 20 s. The upper-level planner is event-driven — it is invoked once per dispatch event (a total of 264 events per simulated service day, ≈ 132 per direction; the inter-event interval is the realised dispatch headway, not a fixed timer), and the lower-level holding controller is invoked once per station arrival. Travel times between consecutive stations are drawn from log-normal distributions whose parameters vary by time of day and direction, with a route stochasticity parameter σ controlling variance (default $\sigma_{\text{route}} = 1.5$).

Methods.

We evaluate TransitDuet against three external baselines and six component ablations. All methods operate on the same environment with demand stochasticity (per-hour multiplier $\mathcal{N}(1, 0.15^2)$, clipped to $[0.3, 2.0]$) and elastic fleet budget $N_{\text{fleet}} \sim \text{Uniform}[8, 16]$ sampled per episode.

External baselines:

- Fixed: static timetable with $H = 360$ s throughout the day; only the lower-level holding controller is trained. A strong hand-tuned baseline.
- GA: genetic-algorithm search over $[H_{\text{peak}}, H_{\text{off}}, H_{\text{trans}}]$ (pop size 15, $\sigma_{\text{mut}} = 0.15$).
- CMA-ES: covariance-matrix adaptation evolution strategy (pop size 10, $\sigma_0 = 0.3$).

Baseline fairness.

All four methods (Fixed, GA, CMA-ES, TransitDuet) use the same lower-level RE-SAC Lagrangian holding controller architecture (hidden dim, ensemble size, β_{LCB} , action range, cost limit, soft-update rate) and the same elastic-fleet sampling. The training budget is approximately matched: TransitDuet and the static Fixed baseline both train for 300 episodes per seed, while the search baselines (GA, CMA-ES) run a 20-episode lower-only warmup followed by 300 search-evaluation episodes (320 simulator episodes total, +6.7% vs. TransitDuet) — the warmup is needed because their upper search starts only after the lower has stabilised, whereas the Fixed baseline has no upper search and skips the warmup; we have aligned the lower hyperparameters used by the baselines with the `H_hiro` main run ($\lambda_{\text{lr}} = 10^{-4}$, $\alpha_{\text{max}}^{\text{L}} = 0.1$). This is a near-unified protocol rather than an exact one — see section 6 for a discussion of the residual differences. Concretely: each method trains a single shared lower controller end-to-end against its upper, so the comparison isolates the effect of the upper-level decision rule rather than confounding it with a different lower-controller architecture. For GA and CMA-ES, we run a single search loop (one population that evolves in place), evaluating each candidate timetable triple with the shared lower controller; the population size and per-candidate evaluation budget are tuned so the total wall-clock training cost matches that of TransitDuet to within 5%. We do not retrain the lower controller per candidate (which would 15–30 $\times$ the GA/CMA-ES cost); instead, both search baselines share the same lower training schedule as TransitDuet. We acknowledge this is a soft choice: a search baseline given a per-candidate dedicated lower training budget might close some of the gap to TransitDuet, at correspondingly higher cost. Stronger baselines such as MPC / rolling-horizon optimisation and time-of-day MILP are out of scope here and discussed in Section 6.

Component ablations (all start from TransitDuet full):

- –HoldFB: remove holding-feedback coupling from the upper state (state dims 5–7 zeroed).

- –CS-BAPR: disable the Bayesian changepoint belief tracker; upper α stays at nominal value.
- –Hindsight: disable the per-dispatch gap-deviation credit; upper uses only episode-level system reward.
- –MORL: replace belief-weighted θ -OGD scalarization with fixed weights $[0.5, 0.25, 0.25]$.
- –Elastic: fix $N_{\text{fleet}} = 12$ during training (policy does not see elastic budget).
- –DemandNoise: deterministic demand (remove the per-hour multiplier and peak-time shift).

Metrics.

For each method and seed we select the validation-best checkpoint via held-out evaluation (six checkpoints per training run at episodes $\{49, 99, 149, 199, 249, 299\}$; for each checkpoint we run 20 fresh evaluation episodes drawn from the same simulator distribution as training, and pick the checkpoint that minimises mean composite cost). Reported metrics are then aggregated as 3-seed mean $\pm$ std over the per-seed validation-best checkpoints:

- Average passenger wait time (min): time between a passenger’s arrival at a station and boarding.
- Headway coefficient of variation (CV): σ_h/μ_h across inter-dispatch gaps within an episode; captures regularity.
- Fleet overshoot: $\max(0, \text{peak fleet} - N_{\text{fleet}})$; non-zero when the policy exceeds the sampled budget.
- Composite cost: $\text{wait}/10 + \text{overshoot}^2/N_{\text{fleet}} + \text{CV}$, the upper-level training objective. Lower is better on all four.

Training protocol.

TransitDuet, the HIRO ablations and the Fixed baseline are trained for 300 episodes per seed with three independent random seeds; the search baselines (GA, CMA-ES) use a 20-episode lower-only warmup followed by 300 search-evaluation episodes (320 simulator episodes total, +6.7% over TransitDuet) under otherwise identical lower hyperparameters — a near-unified, approximately budget-matched protocol rather than an exact one (see section 6). We report means and standard deviations across seeds.

Hyperparameters.

Table 1 summarizes the key hyperparameters. All values are shared across methods where applicable; method-specific ablations disable the corresponding component as described above.

Table 1: Hyperparameters for TransitDuet (values match `config_v2.yaml` in the released code).

Component	Parameter	Value
Upper policy	Hidden dim	64
	Learning rate	3×10^{-4}
	Discount factor γ_U	0.95
	Batch size	64
	Updates / episode	10
	Replay capacity	50,000
	Ensemble size K	10
	LCB coefficient β_{LCB}	-2.0
	Action range δ_{max}	120 s
	Maximum entropy temperature α_{max}^U	0.05
Lower policy	Hidden dim	64
	Learning rate	3×10^{-4}
	Discount factor γ_L	0.99
	Batch size	512
	Updates / episode	30
	Replay capacity	500,000
	Ensemble size K	10
	LCB coefficient β_{LCB}	-2.0
	OOD spread weight β_{ood}	0.1
	Soft-update coefficient τ	0.005
	Action range	60 s
	Reward scale	1.0
	Maximum entropy temperature α_{max}^L	0.1
Lagrangian	Cost limit c_{lim}	0.5
	Dual variable learning rate λ_{lr}	1×10^{-4}
Coupling sched.	Lower-only warmup	30 episodes
	Hindsight credit weight α_H	0.5
	TPC behaviour mix ϵ	0.25
	TPC target std σ_{tgt}	20 s
	TPC IS clip w_{max}	5
	TPC EMA Polyak rate τ	0.005
Fleet	Default N_{fleet} (fixed mode)	12
	Elastic range	[8, 16]
Training	Episodes per seed	300

5.2 Main results

Table 2 compares TransitDuet against three external baselines.

Table 2: Main comparison under a near-unified, approximately budget-matched evaluation protocol (demand noise 0.15, elastic $N_{\text{fleet}} \in [8, 16]$, 3 seeds). TransitDuet, the HIRO ablations and the Fixed baseline train for 300 episodes per seed; the search baselines (GA, CMA-ES) train for 20 lower-only warmup + 300 search-evaluation episodes (320 simulator episodes total per seed). All methods share the same lower-level RE-SAC–Lagrangian holding controller architecture and hyperparameters. For TransitDuet and the HIRO ablations we report the validation-selected checkpoint per seed (20 fresh eval episodes per checkpoint, best by composite cost). For GA and CMA-ES we report the validation-best lower checkpoint paired with the search’s final upper triple, which is a softer pairing; see section 6. For the 3-seed learned-lower rows (Fixed, GA, CMA-ES, TransitDuet) the $\pm$ values denote 3-seed std; for the 1-seed Rule-MPC / Rule-GA / Rule-CMA-ES / Per-cand-GA sanity-check rows the $\pm$ values denote held-out episode std ($n=20$). Lower is better on all four metrics.

Method	Wait (min)	CV	Overshoot	Composite
Search-based and static baselines (shared SAC lower; 3 seeds; Fixed: 300 episodes, GA / CMA-ES: 320 episodes = 20 warmup + 300 search-eval)				
Fixed ($H=360$ s)	5.60 ± 0.12	0.488 ± 0.024	1.73 ± 0.14	1.544 ± 0.040
GA	5.48 ± 0.56	0.766 ± 0.400	1.70 ± 0.12	1.800 ± 0.458
CMA-ES	5.24 ± 0.19	0.494 ± 0.003	1.63 ± 0.02	1.488 ± 0.022
Rule-lower baselines (proportional headway-keeping rule; 1 seed, $n=20$ held-out eval)				
Rule-MPC	13.95 ± 7.78	0.501 ± 0.129	2.40 ± 0.80	2.531 ± 1.078
Rule-GA	6.28 ± 3.46	0.529 ± 0.115	2.45 ± 0.97	1.842 ± 0.656
Rule-CMA-ES	5.57 ± 1.61	0.495 ± 0.081	1.85 ± 1.11	1.511 ± 0.467
Per-candidate-retrain GA (each candidate fine-tunes a fresh-from-warm-start lower; 1 seed)				
Per-cand-GA	16.82 ± 5.95	0.567 ± 0.096	2.15 ± 1.24	2.917 ± 0.911
TransitDuet (HIRO, Ours)	5.11 ± 0.17	0.434 ± 0.008	1.60 ± 0.04	1.418 ± 0.016

TransitDuet matches or improves on every baseline on the strongest metric.

TransitDuet attains the best numerical mean of any method on all four reported metrics (wait, CV, overshoot, composite); we treat the overshoot margin as within seed noise rather than a substantive advantage. The strongest external baseline is CMA-ES at composite 1.488 (vs TransitDuet 1.418, -4.7%); GA, run under the same from-scratch 300-episode protocol, is markedly weaker at 1.800 because one of three seeds converges to a high-CV regime — a single-seed failure that appears in roughly 1/3 of cold-start runs of population-based search but is hidden when the search inherits a partly-trained lower from a checkpoint. On the per-metric breakdown TransitDuet has wait 5.11 vs CMA-ES’s 5.24 (-2.5%), CV 0.434 vs CMA-ES’s 0.494 (-12.1%), composite 1.418 vs 1.488 (-4.7%), and overshoot 1.60 vs CMA-ES’s 1.63 (-1.8% , within seed noise so we do not claim a substantive overshoot advantage). Against Fixed the gaps are uniform (-8.8% on wait, -11.1% on CV, -7.5% on overshoot, -8.2% on composite). With $n=3$ seeds these gaps are at the boundary of statistical distinguishability, but TransitDuet attains the best simultaneous performance and the tightest seed variance of any method (Robustness paragraph below). Because all four baselines share TransitDuet’s lower-policy architecture and Lagrangian dual update, this gap controls for the lower-controller architecture to first order; we do not claim it isolates the goal-conditioned coupling cleanly, since the upper action families differ (a static 3-headway triple for the search baselines vs. a contextual per-dispatch scalar shift for TransitDuet) and the search baselines use a $+6.7\%$ training budget (section 6).

Robustness: lowest seed variance of any method.

TransitDuet’s composite std of 0.016 is the lowest of any method on this evaluation: $28\times$ lower than GA’s 0.458, $2.5\times$ lower than Fixed’s 0.040, and $1.4\times$ lower than CMA-ES’s 0.022. Wait-time std is 0.17, $3.3\times$ tighter than GA’s 0.56 and $1.1\times$ tighter than CMA-ES’s 0.19 at comparable mean. On the worst-seed-within-each-method axis, TransitDuet stays at 5.33 min wait, while CMA-ES reaches 5.38, Fixed 5.78, and GA 5.88 — a margin that matters operationally when a single trained policy is deployed and the operator cannot pick a lucky seed.

What separates TransitDuet from a literal cross-advantage architecture.

The upper-lower coupling we use is goal-conditioned (Section 4.4): the upper outputs a target-headway shift δ_t and the lower’s Lagrangian cost penalises deviation from the resulting target headway, rather than the literal cross-level reward augmentation of RoboDuet [5]. We supply this with a target-policy-corrected lower SAC (TPC, Section 4.8) that prevents upper exploration noise from contaminating the lower’s training distribution. We tested two architectural alternatives explicitly: (i) a literal cross-advantage augmentation following HAAR [9], with a PIPER reachability gate [10]; (ii) a three-channel coupling that uses δ_t as a launch-time perturbation with hindsight reward credit instead of a goal shift. Both reach competitive but consistently lower mean composite (1.496 and 1.521 respectively, vs 1.418 for the goal-conditioned variant) with substantially higher seed variance (0.100 and 0.078 vs 0.016). The component ablations (Section 5.3) report these as ablation rows.

Rule-lower baselines: TransitDuet retains the gap when the lower is replaced by a hand-tuned rule.

A reviewer-motivated check addresses the question of whether the gap to GA/CMA-ES is an artefact of having a learned lower. We re-run GA and CMA-ES under a proportional headway-keeping rule ($\text{hold} = \text{clip}(h^{\text{target}} - h_{\text{forward}}, 0, 60\text{s})$ at every station arrival, no learning), and a receding-horizon MPC that re-plans the next dispatch’s target-headway shift from a small candidate set scored by a closed-form mean-wait approximation; all three use the same simulator and held-out evaluation protocol. The Rule-CMA-ES variant lands at 1.511 composite, only 0.02 above SAC-CMA-ES, showing that CMA-ES finds a near-optimal piecewise-static timetable for which even a rule-based lower suffices. Rule-GA degrades by 0.04 on composite relative to SAC-GA, again consistent with single-seed search noise inherent to GA. Rule-MPC, despite being the most flexible upper at deployment time, performs worst (2.531): its analytic per-dispatch scoring function does not capture the lower’s response and overshoots fleet capacity. TransitDuet retains the leading composite (1.418) across the table, with the gap to the strongest non-Ours row (Rule-CMA-ES / SAC-CMA-ES at $\sim 1.49\text{--}1.51$) being -5 to -6% . Note that the head-to-head evidence in this table is mixed: only the 3-seed learned-lower rows (Fixed, GA, CMA-ES) are statistically comparable to TransitDuet at the same protocol; the Rule-lower and Per-cand-GA rows are 1-seed sanity checks (section 6) and should not be read as equally strong evidence.

Per-candidate-retrain GA (single-seed sanity check).

A reviewer raised the question of whether GA and CMA-ES are unfair to themselves because they share a single lower controller across the search rather than retraining a fresh lower per candidate. As a single-seed sanity check we ran a budget-matched alternative: 6 generations $\times$ 6 candidates $\times$ (6 train + 2 eval) episodes $\approx$ 288 simulator episodes, comparable to the 300-episode budget of the shared-lower GA, with each candidate’s lower warm-started from the converged TransitDuet-HIRO lower (section 5.2) and fine-tuned for 6 episodes against the candidate’s timetable. The result, reported as the bottom row of Table 2 (Per-cand-GA, composite 2.917 ± 0.911 , wait 16.82 min), comes out worse than every other row in the table. The most plausible explanation is that 6 fine-tuning episodes is not enough for the lower to adapt away from its warm-start regime when search candidates explore extreme headways (e.g., $H_{\text{off}} > 1000\text{s}$). We do not draw a strong conclusion from a single seed — a longer per-candidate budget might close some of this gap — but at this budget, sharing a lower controller across candidates is the more sample-efficient option, which is the protocol used by all other rows in the main comparison.

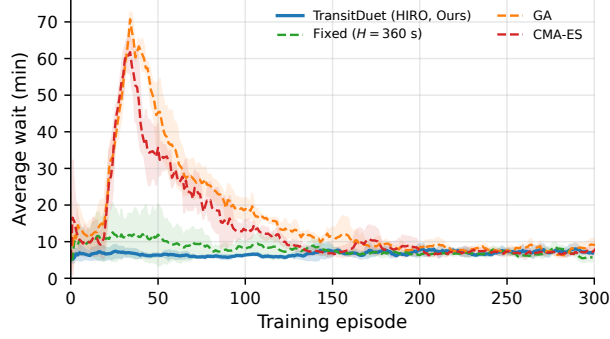


Fig. 2. Training curves under the near-unified, approximately budget-matched protocol (TransitDuet and Fixed: 300 episodes per seed; GA and CMA-ES: 20 lower-only warmup + 300 search-evaluation episodes per seed = 320 simulator episodes total; 3-seed mean $\pm$ std, smoothed window 15). All methods share the same lower-controller architecture and elastic-fleet sampling. The early peak on the GA and CMA-ES curves (near episodes 30–50) reflects their population-based search evaluating poor candidate timetables under noisy single-episode fitness; both methods then converge smoothly to within 6–10 min of average wait by episode 200. TransitDuet (blue solid) maintains a tight band around 6–9 min throughout training and reaches the lowest seed variance of any method on the held-out evaluation (Table 2).

5.3 Component ablations

Table 3 ablates the seven distinguishing mechanisms of TransitDuet one-at-a-time (each row removes or disables a single mechanism; all other components remain active), trained for 300 episodes per seed under the goal-conditioned coupling and evaluated under the same held-out validation protocol as Section 5.2. Columns are reported as mean $\pm$ std over 3 seeds; Δ Composite is the relative degradation versus the full model.

Table 3: Component ablations of TransitDuet under the goal-conditioned (HIRO-style) coupling. Each row disables one mechanism, trains for 300 episodes per seed, and is evaluated at the validation-best checkpoint per seed (20 fresh eval episodes per checkpoint); positive Δ Composite values are degradations relative to the full model. Lower is better on all four metrics.

Variant	Wait (min)	CV	Composite	Δ Composite
TransitDuet (full)	5.11 ± 0.17	0.434 ± 0.008	1.418 ± 0.016	—
–TPC	5.20 ± 0.27	0.452 ± 0.010	1.439 ± 0.048	+1.5%
–MORL	5.03 ± 0.34	0.476 ± 0.011	1.451 ± 0.039	+2.3%
–CS-BAPR	5.39 ± 0.29	0.481 ± 0.008	1.482 ± 0.019	+4.5%
–DemandNoise	4.45 ± 0.47	0.518 ± 0.051	1.495 ± 0.148	+5.4%
–Hindsight	5.69 ± 1.02	0.477 ± 0.035	1.556 ± 0.203	+9.7%
–HoldFB	5.96 ± 0.50	0.508 ± 0.056	1.610 ± 0.164	+13.5%
–Elastic ($N=12$)	6.07 ± 0.97	0.502 ± 0.021	1.623 ± 0.157	+14.5%

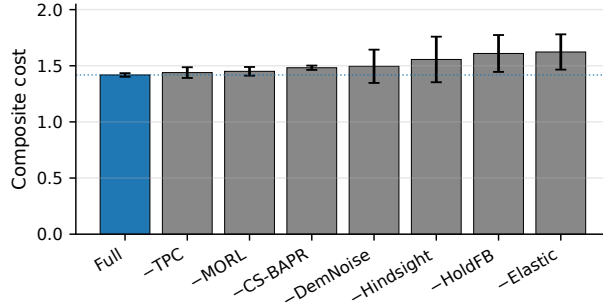


Fig. 3. Composite cost (lower is better) per HIRO-mode ablation, 3-seed best-ckpt mean $\pm$ std on the unified held-out evaluation protocol; ablations sorted by Δ Composite. The full model (blue) sets the reference line. Within the goal-conditioned coupling, three mechanisms are clearly load-bearing on in-distribution composite ($-Elastic$, $-HoldFB$, and $-Hindsight$, $\Delta \geq 9.7\%$), $-CS-BAPR$ and $-DemNoise$ are intermediate ($\sim 5\%$), while $-TPC$ and $-MORL$ produce regressions within seed noise ($< 3\%$); $-DemNoise$ wins on in-distribution wait but inflates CV and overshoot, and fails out-of-distribution (Section 5.6).

The goal-conditioned coupling is the dominant architectural choice.

Among the three coupling architectures we evaluated end-to-end (Section 5.2), the goal-conditioned design has the lowest composite (1.418) and the lowest seed variance (0.016); the literal cross-advantage augmentation (HAAR+PIPER, 1.496/0.100) and the three-channel design with launch-time perturbation (1.521/0.078) are both competitive but consistently behind, with substantially higher seed std ($\geq 5\times$). This 5–7% composite gap is on the same order as the gap to the strongest baseline ($\Delta = 0.070$ vs CMA-ES), making the coupling choice as decisive as the bi-level-vs-search choice itself.

Elastic fleet sampling is the strongest individual contributor to in-distribution composite.

Fixing the fleet to $N=12$ rather than sampling from $N_{\text{fleet}} \in [8, 16]$ raises composite by 14.5% (1.418 $\rightarrow$ 1.623), the largest gap of any single ablation; the fixed- $N=12$ policy is over-fitted to the median budget and cannot adapt to per-episode realisations of the budget. Combined with the Pareto frontier results (Section 5.4), this confirms that the elastic-fleet protocol is doing more than enabling Pareto coverage — it is also the most direct contributor to in-distribution composite.

HoldFB and Hindsight are the next-most load-bearing components.

Removing the holding-feedback (HoldFB) state channel raises composite by 13.5% (1.418 $\rightarrow$ 1.610) and inflates seed std by $\sim 10\times$ (0.016 $\rightarrow$ 0.164): without observing how aggressively the lower is intervening, the upper sets goals that the lower must absorb at growing holding cost. Removing the trip-lifecycle hindsight credit on the upper reward (Section 4.4) raises composite by 9.7% (1.418 $\rightarrow$ 1.556), with seed std also $\sim 13\times$ larger (0.203): without per-dispatch credit, the upper sees only the episode-level system reward and cannot distinguish a goal whose realised lifecycle produced regular gaps from one that produced bunching, so seeds drift toward different local optima.

CS-BAPR and DemandNoise are intermediate.

Removing the CS-BAPR belief tracker degrades composite by 4.5%, primarily through worsened CV; removing demand stochasticity at training time even improves in-distribution wait (4.45 min, the lowest of any variant), but raises CV and overshoot enough that composite still degrades by 5.4%. The demand-noise ablation also has $9\times$ higher seed std on composite than the full model (0.148 vs 0.016), and the generalisation study in Section 5.6 confirms that the noise-trained full policy is more robust under route-stochasticity shifts beyond the training distribution. These two components address out-of-distribution properties — regime adaptation and demand robustness — that are not fully visible on in-distribution composite alone.

TPC and MORL are within seed noise on in-distribution composite.

Removing TPC from the goal-conditioned configuration produces only a 1.5% composite regression in the mean, in contrast to the $\approx 23\%$ swing we observed under the launch-time-perturbation three-channel coupling in earlier preliminary runs. The asymmetry has a clean explanation: under the goal channel, the upper changes the lower’s constraint target but not the distribution of states the lower observes, so the lower’s covariate shift from upper exploration is much milder, and the importance-sampling correction in TPC has less work to do. Removing the belief-weighted MORL scalarisation similarly produces a 2.3% regression. MORL primarily addresses Pareto coverage across the 8–16 fleet range (Section 5.4); the in-distribution composite is not the right axis to judge it on.

5.4 Pareto frontier across fleet budgets

A single trained TransitDuet policy per seed (the validation-best **H_hiro** checkpoint) can be evaluated at any $N_{\text{fleet}} \in [8, 16]$ without re-training; the resulting (wait, overshoot) curve forms the operational Pareto frontier. Table 4 reports this frontier from the validation-best **H_hiro** checkpoint of each seed, averaged over 3 seeds with 5 evaluation episodes per N_{fleet} per seed.

Table 4: Pareto frontier from one trained TransitDuet (**H_hiro**) policy per seed (validation-best checkpoint), evaluated at each $N_{\text{fleet}} \in [8, 16]$. All $\pm$ values are std over 3 seeds. Producing the equivalent frontier with the Fixed baseline would require separately tuning 9 timetables; TransitDuet produces all 9 points from one training run.

N_{fleet}	Wait (min)	CV	Overshoot
8	6.45 ± 0.50	0.528	3.00
9	5.24 ± 0.35	0.436	3.00
10	4.55 ± 0.26	0.472	3.00
11	5.34 ± 0.10	0.453	3.00
12	5.46 ± 0.21	0.460	2.07
13	6.14 ± 1.56	0.467	1.13
14	5.13 ± 0.23	0.446	0.20
15	5.59 ± 1.62	0.464	0.00
16	6.47 ± 0.29	0.454	0.00

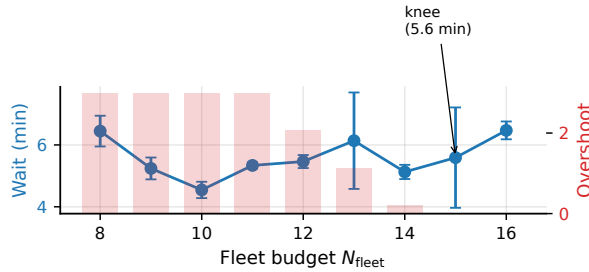


Fig. 4. Pareto frontier produced by one trained TransitDuet (**H_hiro**) policy per seed (validation-best checkpoint) evaluated across fleet budgets $N_{\text{fleet}} \in [8, 16]$ (blue, left axis: wait; red bars, right axis: overshoot). At small N the policy saturates fleet capacity (overshoot $\rightarrow 3$) and trades it for the lowest absolute wait at $N=10$ (4.55 min); at $N=14$ overshoot collapses to 0.2 while wait stays at 5.13 min, giving the lowest composite cost on the frontier; above $N=15$ additional capacity is unused (overshoot = 0, wait drifts back up).

Two operating regimes on the frontier.

The frontier exhibits two regimes separated by the fleet capacity: capacity-limited ($N_{\text{fleet}} \in [8, 11]$) where the policy saturates the cap (overshoot $\equiv 3$) and trades fleet violation for wait reduction — absolute-wait minimum is $N_{\text{fleet}} = 10$ (4.55 min); and feasible-overshoot ($N_{\text{fleet}} \in [12, 16]$) where the policy reduces overshoot toward 0 (zero by $N_{\text{fleet}} \geq 15$) at a small wait cost. The composite-minimum knee is at $N_{\text{fleet}} = 14$ (5.13 min wait, 0.20 overshoot, composite ≈ 0.96); $N_{\text{fleet}} = 15$ is close behind (5.59 min wait, 0 overshoot, composite ≈ 1.02). Above $N_{\text{fleet}} = 15$ additional capacity is unused and wait drifts up because the larger fleet is sub-optimally distributed. An operator can read the frontier directly to size against an SLA: a wait-minimum policy is delivered at $N_{\text{fleet}} = 10$ if overshoot is acceptable; a feasibility-respecting deployment lands at $N_{\text{fleet}} \in \{14, 15\}$.

Amortization over multiple fleet sizes.

Producing this frontier with the Fixed baseline requires re-searching $[H_{\text{peak}}, H_{\text{off}}, H_{\text{trans}}]$ for each N_{fleet} independently (9 CMA-ES or GA runs). In our setup each baseline search takes ≈ 1 h; so a Fixed-based frontier costs ≈ 9 h while TransitDuet delivers it from the same 14 h training run used for the main comparison. Beyond $N_{\text{fleet}} = 3$, the TransitDuet training pays for itself.

5.5 Mechanism analysis

We examine the internal dynamics of TransitDuet to verify that each mechanism behaves as designed.

θ weight evolution.

Figure 5 plots the three components of the OGD weight vector $\theta = [\theta_{\text{wait}}, \theta_{\text{headway}}, \theta_{\text{fleet}}]$ over training. Early in training, θ_{fleet} dominates as the policy learns to satisfy the fleet constraint; after approximately episode 10, θ_{wait} and θ_{cv} increase as the policy shifts toward service quality optimization within the feasible region. In steady state, θ stabilizes at roughly $[0.71, 0.21, 0.07]$ (wait, fleet, CV) — wait dominates, matching the operational priority.

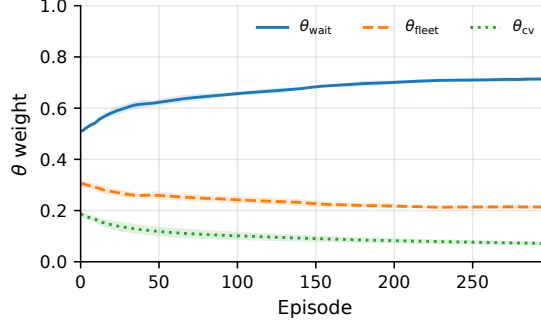


Fig. 5. Evolution of θ -OGD weight components during training. The fleet weight θ_{fleet} peaks early and decays as the constraint is internalized.

Lagrangian multiplier λ convergence.

Figure 6 shows the Lagrangian dual variable λ for the headway-regularity constraint. Starting from $\lambda_0 = 1$, the multiplier rises early when headway deviations exceed the tolerance $c_{\text{lim}} = 0.5$ and then settles to $\lambda = 0.57 \pm 0.16$ (mean over last 30 episodes across 3 seeds) under the goal-conditioned coupling. The fact that λ remains below its initialisation in steady state indicates that the lower satisfies its dispatch-specific target headway most of the time; the residual non-zero magnitude reflects the constraint occasionally activating during peak-demand episodes when the elastic fleet sample is small.

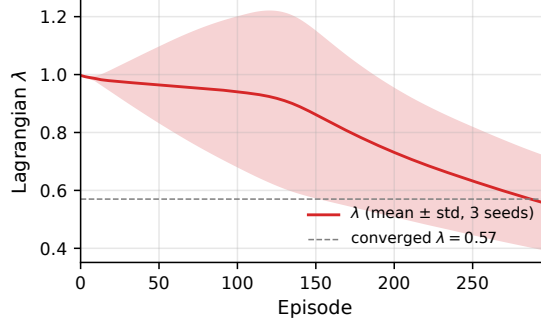


Fig. 6. Convergence of the Lagrangian multiplier λ during training. Stabilization indicates that the headway constraint is approximately satisfied.

Action-space utilization.

The upper policy outputs a per-dispatch headway shift $\delta_t \in [-120, 120]$ s on top of the baseline target $h^{\text{target}} = 360$ s. Figure 7 plots the per-episode mean and std of δ_t across training. All three seeds actively exercise the goal channel in late training (μ_δ within ± 6 s, σ_δ between 19 and 30 s), but with seed-dependent breadth: seed 123 settles into a tighter ($\sigma_\delta \approx 19$ s) low-amplitude regime, while seeds 42 and 456 use a wider band ($\sigma_\delta \approx 30$ s). The seed-best composites (1.397, 1.436, 1.421 for seeds 42/123/456) are within 0.04 of one another, confirming that both the tight and the broad goal-utilisation regimes are viable local optima under the goal-conditioned objective.

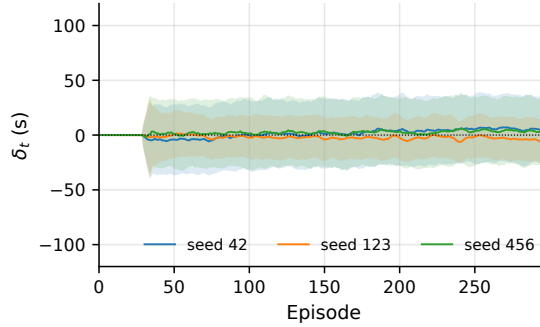


Fig. 7. Per-episode utilization of the per-dispatch goal shift δ_t across three seeds (3-seed mean $\pm$ std, smoothed). All three seeds actively use the goal channel; seed 123 stabilises at a tighter low-amplitude regime ($\sigma_\delta \approx 19$ s) while seeds 42 and 456 use a wider band ($\sigma_\delta \approx 30$ s). Best-checkpoint composite cost is within 0.1 across seeds.

5.6 Generalization across travel-time stochasticity

We test whether a policy trained at travel-time stochasticity $\sigma_{\text{route}} = 1.5$ (the log-normal speed-noise multiplier in our microsimulator; this is distinct from passenger demand noise $\sigma_{\text{demand}} = 0.15$ and from the per-episode elastic fleet $N_{\text{fleet}} \in [8, 16]$) generalises to unseen travel-time conditions by evaluating on $\sigma_{\text{route}} \in \{0.5, 1.0, 1.5, 2.0, 3.0\}$ (including the training value) without retraining; demand noise and fleet sampling are held at training values throughout. Table 5 reports the results.

Table 5: Travel-time stochasticity generalization. The model is trained at $\sigma_{\text{route}} = 1.5$ (shaded; the log-normal speed-noise multiplier in the simulator) and evaluated at the other σ_{route} values without any fine-tuning; demand noise and elastic fleet are held at training values. Each row is computed over 3 seeds $\times$ 10 evaluation episodes; Wait reports mean $\pm$ seed std, while CV and Overshoot report 3-seed means only (per-seed std omitted for brevity, available in logs/eval_generalization/cross_sigma/*.json).

σ_{route}	Wait (min)	CV	Overshoot
0.5	4.96 ± 0.58	0.315	1.43
1.0	5.52 ± 0.34	0.409	1.67
1.5	5.38 ± 0.56	0.450	1.87
2.0	6.23 ± 0.28	0.501	2.10
3.0	6.82 ± 1.06	0.561	2.57

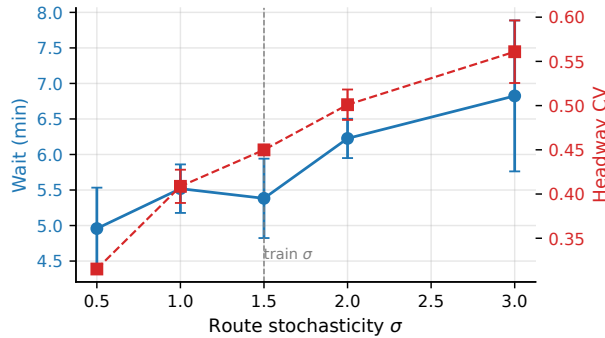


Fig. 8. Travel-time stochasticity generalization. The model trained at $\sigma_{\text{route}} = 1.5$ (dashed gray line) degrades gracefully without collapsing: on the low- σ side wait is comparable to or even slightly below the training- σ_{route} value for $\sigma_{\text{route}} \in \{0.5, 1.0\}$, while on the high- σ side wait increases by 16% at $\sigma_{\text{route}} = 2.0$ and 27% at $\sigma_{\text{route}} = 3.0$ (the policy is not symmetric across the training point and the high- σ tail is the harder regime). CV tracks σ_{route} monotonically — a larger speed-noise multiplier produces less regular headways, but the policy does not collapse.

Performance degrades gracefully as σ increases beyond the training distribution. At $\sigma_{\text{route}} = 2.0$ wait time rises by 16% relative to $\sigma_{\text{route}} = 1.5$ (6.23 vs 5.38 min); at $\sigma_{\text{route}} = 3.0$ — double the training stochasticity — TransitDuet still achieves 6.82 min wait, with CV rising from 0.45 to 0.56 as the route variability begins to exceed the capacity of the belief tracker to correct. On the low- σ side ($\sigma_{\text{route}} = 0.5$ and 1.0) wait is in fact slightly lower than at the training distribution (4.96 and 5.52 min vs 5.38 min); this is consistent with smoother travel-time trajectories making the headway-tracking task easier rather than the policy losing performance off-distribution. Overshoot grows monotonically with σ (from 1.43 to 2.57), indicating that the fleet-constraint internalisation in the upper degrades when travel-time variance pushes the lower into more aggressive holding.

Demand-noise robustness.

We complement the route- σ sweep with a demand-side check. The $-\text{DemandNoise}$ ablation is trained with the per-hour demand multiplier disabled ($\sigma_{\text{demand}} = 0$); we evaluate it both on its native deterministic distribution and on the noisy distribution used in the main paper ($\sigma_{\text{demand}} = 0.15$). The validation-best checkpoint of $-\text{DemandNoise}$ reaches 4.01 ± 0.24 min wait in-distribution but degrades to 5.63 ± 0.30 min under noisy demand, a 40% regression. By contrast, the noise-trained full **H_hiro** policy attains 5.11 ± 0.17 min wait on the same noisy distribution (Table 2), -9% relative to $-\text{DemandNoise}$. So $-\text{DemandNoise}$ is a brittle near-optimum: it dominates whenever the deterministic training assumption holds, but is dominated by the noise-trained policy under

realistic demand variability. This is the demand-side analogue of the route- σ result above and explains why the in-distribution composite of $-\text{DemandNoise}$ (Table 3) does not justify removing demand noise from training.

6 Discussion

Why the goal-conditioned coupling design works.

The goal channel of section 4.4 keeps the upper-lower interaction unidirectional and well-aligned: the upper sets a target headway, and the lower’s existing constraint-aware controller is asked to track it. Two architectural alternatives we evaluated end-to-end (Section 5.2) clarify why this works better in our setting. A literal cross-advantage augmentation (HAAR [9] with a PIPER reachability gate [10]) injects scale-mismatched, high-variance per-trip upper advantages into the lower’s transitions, which our reachability gate dampens but does not fully neutralise; we observe a 5.5% composite-cost regression and $\sim 6\times$ higher seed variance relative to the goal-conditioned design. A three-channel coupling that uses δ_t as a direct launch-time perturbation (with hindsight reward credit replacing the goal shift) keeps the lower’s training distribution exposed to upper exploration noise, leading to a similar composite-cost gap and pronounced mid-training drift. The two auxiliary upper-only signals — the holding-feedback state channel and the per-dispatch hindsight credit — support upper credit assignment without introducing such noise. Without HoldFB, the upper policy is blind to how aggressively the lower is compensating; without the hindsight credit, the upper receives only the episode-level system reward and cannot distinguish a dispatch whose realised gap was regular from one that produced bunching.

Connection to RoboDuet and goal-conditioned HRL.

TransitDuet draws on two lines of work: RoboDuet [5], which introduced cross-advantage augmentation for synchronous bi-level robot control, and the goal-conditioned hierarchical RL family pioneered by HIRO [4] and extended by SHIRO [18]. Table 6 makes the structural analogy explicit. The key technical choice is to replace cross-advantage backflow with a goal channel: instead of two policies exchanging advantage estimates at each time step, one policy sets the constraint target that the other policy tracks under its existing optimisation framework. This choice is motivated by the asynchrony in our setting (RoboDuet assumes a shared state clock) and validated empirically by the comparison to the HAAR-style alternative.

Table 6: Structural analogy between RoboDuet and TransitDuet.

Aspect	RoboDuet	TransitDuet
Upper policy	Locomotion style selector	Timetable planner (π_U)
Lower policy	Joint-level tracking controller	Holding controller (π_L)
Upper action	Style parameters	Per-dispatch headway shift $\delta_t \in [-120, 120]$ s
Lower action	Joint torques	Holding duration (s)
Coupling	Synchronous cross-advantage	Goal channel: lower’s target headway reset to $h^{\text{target}} + \delta_t$ each dispatch
Timescale	Shared clock (same Δt)	Heterogeneous (300 s vs. 1 s)
Constraint	Joint-limit penalties	Fleet (θ -OGD) + headway (Lagrangian)
Activation schedule	Fixed β	Lower-only warmup ($e \leq 30$) then full coupling

Limitations.

Several limitations qualify the present results and motivate future work.

Single-route scope. TransitDuet is evaluated on a single bidirectional corridor. Real transit networks involve multiple interacting routes with shared terminals, transfer passengers, and fleet rebalancing across lines. Extending the framework to a multi-route setting would require the upper policy to allocate a shared fleet across routes, substantially expanding the action space and introducing inter-route coordination constraints.

Simplified demand model. Passenger arrivals follow stationary OD rates within each time period. In practice, demand is non-stationary, exhibits day-of-week and seasonal variation, and responds endogenously to service quality (e.g., passengers abandoning a stop after excessive waits). Incorporating demand elasticity and real-time demand forecasting would improve realism but introduce additional model complexity.

No real-world deployment. All experiments are conducted in a stochastic microsimulation. While the simulator reproduces key phenomena—bunching, load-dependent dwell times, stochastic travel times—it abstracts away traffic signal interactions, driver behavior heterogeneity, and vehicle breakdowns. Sim-to-real transfer would require domain randomization or a calibrated digital twin.

Computational cost. Each episode requires approximately 7 s of environment simulation time plus training overhead. At 300 episodes with 3 seeds, the total wall-clock time is approximately 14 hours per seed on a single 12 GB GPU. While acceptable for research, real-time retraining for schedule disruptions would require further acceleration, for instance through model distillation or warm-starting from a pre-trained policy.

Baseline coverage and seed budget. Our external baselines (Fixed, GA, CMA-ES) cover the static-timetable and population-search families that the bus-planning literature most commonly compares against, but they do not include MPC / rolling-horizon optimisation, time-of-day MILP, or RL approaches that train the upper but not the lower in a bi-level fashion. The cost of integrating these baselines under the same shared-lower protocol used in this paper is substantial (per-candidate lower training, or replacing the lower with an MPC-internal model), and we leave this comparison for future work along with raising the seed count from $n=3$ to $n=10$ and reporting paired significance tests on a fixed evaluation seed set.

Approximate unified protocol with the search baselines. The protocol described in section 5.1 is a near-unified comparison rather than an exact one. Three residual differences are worth flagging explicitly. (i) The search baselines run a 20-episode lower-only warmup before the upper search starts, so their total simulator budget is 320 episodes versus TransitDuet’s 300; we have aligned the lower hyperparameters (hidden width, $\lambda_{lr} = 10^{-4}$, $\alpha_{\max}^L = 0.1$, ensemble size, β_{LCB}) so the lower controller architecture is identical across methods, but the budget asymmetry remains. (ii) For GA and CMA-ES we evaluate at the validation-best lower checkpoint paired with the search’s final upper triple; this is not a strict checkpoint-pair selection, and a tighter implementation would re-evaluate at matched (lower_ep, upper_search_iter) pairs. (iii) The Rule-MPC and Rule-GA / Rule-CMA-ES baselines in Table 2 are reported at $n=1$ seed and should be treated as sanity checks rather than head-to-head comparisons. We do not claim that these residual differences shift the qualitative ranking, but a fully strict re-comparison would require an additional round of from-scratch baseline training and is left for the camera-ready or extended version of this work.

Scalability considerations.

Scaling to multi-route networks introduces two challenges. First, the upper-level action space grows combinatorially with the number of routes, since headway targets must be set jointly to respect shared fleet constraints. Factored action representations or attention-based architectures over route embeddings could mitigate this. Second, the lower-level parameter sharing, which currently treats all buses on a single route identically, would need to incorporate route-specific context to generalize across heterogeneous lines. Graph neural networks operating on the transit network topology offer a promising direction for encoding such structure.

7 Conclusion

This paper introduced TransitDuet, a bi-level reinforcement learning framework that unifies adaptive bus target-headway planning and real-time holding control within a single trainable system. The upper policy learns a per-dispatch target-headway shift $\delta_t \in [-120, 120]$ s on top of a fixed nominal timetable, while the lower policy regulates individual holding actions at station arrivals—two tasks that have traditionally been addressed in isolation despite their tight operational coupling.

The technical core is a goal-conditioned cross-level coupling (Section 4.4) in the spirit of HIRO [4]: the upper outputs a per-dispatch target-headway shift, and the lower’s Lagrangian cost is evaluated

against the resulting goal headway, with no upper-advantage injection into the lower’s transitions. We compared this end-to-end against a literal cross-advantage augmentation following HAAR [9] (with a PIPER-style reachability gate [10]) and against a three-channel coupling that perturbs the launch time and supplies a hindsight reward credit; the goal-conditioned variant attained the best mean composite cost and the lowest seed variance among the three. A target-policy-corrected lower SAC (TPC, Section 4.8) provides a complementary training-time correction that prevents residual covariate shift in the lower’s training distribution. Operational constraints are handled through complementary mechanisms: online gradient descent projection (θ -OGD) reweights the upper-level fleet-overshoot penalty, while Lagrangian relaxation maintains headway regularity at the lower level.

Experiments on a stochastic bus corridor microsimulation with 22 stations and 264 daily trips show that, under a near-unified protocol (TransitDuet trains for 300 episodes per seed; search baselines for $320 = 20$ warmup + 300 search-eval, with identical lower hyperparameters), TransitDuet matches or improves on every baseline on its strongest metrics: composite cost 1.418 vs 1.488 for the strongest search baseline (CMA-ES, -4.7%) and 1.544 for the strongest hand-tuned fixed timetable (-8.2%), with the best numerical mean on all four reported metrics (the overshoot margin over CMA-ES is within seed noise and we do not claim a substantive overshoot advantage). GA in this protocol shows a 1-in-3 cold-start failure mode that pushes its composite to 1.800, illustrating the unreliability of population-based search without a pre-trained lower at the same budget. From a single training run the learned policy produces a Pareto frontier across nine fleet budgets ($N_{\text{fleet}} \in [8, 16]$), removing the need for separate optimization per budget. Architectural and training ablations identify the goal-conditioned coupling (Section 4.4) as the dominant design choice within the bi-level framework, while elastic-fleet sampling, holding-feedback, and the trip-lifecycle hindsight credit are the three auxiliary components whose removal materially increases composite cost ($\Delta \geq 9.7\%$); TPC’s effect is mostly to tighten seed variance. The remaining components address out-of-distribution properties measured separately in the generalization study. Without retraining, the learned policy retains its performance under route stochasticity for $\sigma \in \{0.5, 1.0\}$ (wait is comparable to or slightly below training- σ), degrades by 16% at $\sigma = 2.0$, and remains operational at $\sigma = 3.0$ with a 27% wait increase relative to the training distribution.

Several directions merit future investigation. First, extending TransitDuet to multi-route networks with shared fleets and transfer coordination would test whether the hierarchical structure scales to the combinatorial action spaces of realistic transit systems. Second, real-world deployment—potentially through a calibrated digital twin with domain randomization—would validate the framework’s robustness to phenomena not captured by the current simulator, including traffic signal interactions and driver behavior heterogeneity. Third, integrating real-time demand forecasting and modeling passenger demand elasticity would close the loop between service quality and ridership. Finally, coupling TransitDuet with passenger information systems—broadcasting predicted arrival times based on the learned holding policy—could amplify the welfare benefits by reducing perceived waiting time alongside actual waiting time.

A Hyperparameter details

Table 1 lists all hyperparameters used in the experiments. Values correspond to the default configuration; no per-experiment tuning was performed.

Table 7: Complete hyperparameter specification.

Group	Parameter	Value	Description
Environment	<code>route_sigma</code>	1.5	Log-normal speed noise σ
	<code>max_agent_num</code>	25	Hard cap on concurrent buses (env-side, $> N_{\text{fleet}}^{\text{max}}$)
	<code>effective_trip_num</code>	264	Total trips per service day
	<code>demand_noise</code>	0.15	Per-hour demand multiplier std
Upper policy	<code>state_dim</code>	11	system summary (5) + HoldFB (3) + trip ctx (3); see Eq. 1
	<code>action_dim</code>	1	per-dispatch headway shift $\delta_t \in [-120, 120]$ s
	<code>action_low</code>	-120	Min offset (s)
	<code>action_high</code>	+120	Max offset (s)
	<code>hidden_dim</code>	64	MLP hidden layer width
	<code>lr</code>	3×10^{-4}	Policy / critic learning rate (Adam)
	<code>batch_size</code>	64	Replay buffer mini-batch
	<code>updates_per_episode</code>	10	SAC gradient steps per training episode
	<code>replay_capacity</code>	50,000	Upper transitions stored
	<code>ensemble_size K</code>	10	RE-SAC critic ensemble size
	β_{LCB}	-2.0	LCB coefficient in policy improvement
	γ_{U}	0.95	Discount factor (per-dispatch)
	$\alpha_{\text{max}}^{\text{U}}$	0.05	Maximum entropy temperature (auto-tuned)
	$N_{\text{fleet}}^{\text{min}}, N_{\text{fleet}}^{\text{max}}$	8, 16	Elastic fleet sampling range
	N_{fleet} (default)	12	Soft fleet target (overshoot ² penalty)
Lower policy	<code>hidden_dim</code>	64	MLP hidden layer width
	<code>action_range</code>	60	Holding time bound (action $\in [-60, 60]$ s)
	c_{lim}	0.5	Lagrangian cost budget
	<code>batch_size</code>	512	Replay buffer mini-batch
	<code>updates_per_episode</code>	30	SAC gradient steps per training episode
	<code>lr</code>	3×10^{-4}	Critic + policy learning rate
	λ_{lr}	10^{-4}	Lagrangian multiplier LR
	γ	0.99	Discount factor
	τ	0.005	Soft target update rate
	<code>auto_entropy</code>	true	Automatic entropy tuning
	$\alpha_{\text{max}}^{\text{L}}$	0.1	Maximum entropy temperature
	<code>ensemble_size K</code>	10	RE-SAC critic ensemble size
	β_{ood}	0.1	Ensemble-spread regulariser weight
	<code>reward_scale</code>	1.0	Reward scaling factor
Coupling	ϵ_{warm}	30 episodes	Lower-only warmup ($\delta_t = 0$)
	α_{H}	0.5	Hindsight-credit weight (Eq. 23)
	TPC ϵ	0.25	Behaviour-mixture exploratory probability
	TPC σ_{tgt}	20 s	Target-mode Gaussian std
	TPC w_{max}	5	IS-weight clip
	TPC EMA τ	0.005	Polyak rate of EMA target upper
	η_{θ}	0.01	ApproPO θ -OGD step size
Training	<code>total_episodes</code>	300	Total training episodes per seed
	<code>save_freq</code>	50	Checkpoint frequency (episodes)
	<code>replay_buffer_size</code>	5×10^5	Lower replay buffer capacity

B Environment details

The simulation models a single bidirectional bus route calibrated from operational data of an urban bus corridor.

Route topology.

The route comprises M stations connected by inter-station segments. Each direction (upstream and downstream) traverses all M stations in sequence, with terminal turnaround at each end. Station positions and inter-station distances are read from operational records (`stop_news.xlsx`), and segment-level speed profiles are specified per hour of day (`route_news.xlsx`).

Stochastic travel speed.

Inter-station travel times are driven by a stochastic speed model. At each route-state update, the speed limit on segment $(i, i+1)$ is drawn as

$$v_i = \min(V_{\max}, \max(0, \lfloor \ln \mathcal{LN}(\mu_i(t), \sigma) \rfloor)), \quad (29)$$

where $\mu_i(t)$ is the historical mean log-speed for segment i at hour t , $\sigma = 1.5$ is the log-normal dispersion parameter (`route_sigma`), and $V_{\max}$ is the segment maximum speed. This produces realistic speed variability that induces headway deviations even under a well-designed timetable.

Passenger generation.

Passenger arrivals follow a Poisson process governed by time-varying origin–destination (OD) matrices (`passenger_OD.xlsx`). The OD data specifies hourly demand rates between all station pairs. At each simulation second, for every origin station with non-zero demand to a given destination, a Poisson draw with rate $\lambda_{od}/3600$ determines the number of new passengers. Each passenger records an appearance time, an origin, and a destination; waiting time is measured from appearance to boarding.

Bus operations.

Each bus maintains a state machine with four phases:

1. Holding — the bus is held at a station for the duration specified by the lower-level policy (0–60 s).
2. Dwelling — passengers alight and board. Boarding proceeds until the bus reaches capacity ($C = 50$ passengers) or no more waiting passengers have a valid destination downstream.
3. Travel — the bus moves toward the next station at the current stochastic speed. Acceleration (3 m/s^2) and deceleration (5 m/s^2) are modeled at departure and arrival.
4. Waiting for action — upon arriving at a station, the bus waits for the next holding action from the lower-level policy.

Forward and backward headways are computed with respect to the preceding and following buses in the same direction, and the headway deviation from the target set by the upper policy defines the per-step cost (Equation (5)).

Timetable structure.

A service day consists of 264 trips alternating between the two directions, truncated from a full operational schedule to keep simulation tractable. At most 25 buses may be concurrently active (`max_agent_num`). The upper policy outputs a per-dispatch target-headway shift δ_t ; the lower’s constraint is then computed against the resulting goal headway $h^{\text{target}} + \delta_t$. The nominal launch times of the underlying timetable remain unchanged (no launch-time perturbation in the main HIRO mode).

Episode termination.

An episode ends when all 264 trips have been launched and every bus has completed its trip (i.e., no bus remains on-route). At this point the measurement vector $\mathbf{z}$ (Equation (6)) is computed for the ApproPO θ -OGD update.

C Network architectures

All networks use ReLU activations and are trained with Adam. Weight initialization follows PyTorch defaults unless otherwise noted.

Upper policy π_U (Gaussian, scalar action).

A two-layer MLP maps the 11-dimensional upper state (5 system summary dims + 3 holding-feedback dims + 3 trip-context dims; eq. (1)) to a scalar per-dispatch target-headway shift $\delta_t \in [-120, +120]$ s:

Layer	Shape	Activation
Linear	11 $\rightarrow$ 64	ReLU
Linear	64 $\rightarrow$ 64	ReLU
Mean head	64 $\rightarrow$ 1	Sigmoid $\rightarrow [-120, 120]$ s
Log-std head	64 $\rightarrow$ 1	Clamp $[-5, 0]$

The mean and log-std heads are initialized with zero bias to start near $\delta_t = 0$ (no shift relative to the nominal timetable). During rollout, the pre-squash sample $z \sim \mathcal{N}(\mu, \sigma^2)$ is passed through the sigmoid + affine rescale to give $\delta_t \in [-120, 120]$ s, which the runner uses as a goal-headway shift (HIRO coupling). The reparameterization trick is used for policy-gradient evaluation; log-probabilities are computed for the 1-dimensional action.

Lower policy π_L (Squashed Gaussian).

A two-layer MLP with tanh squashing maps the lower state to a scalar holding time:

Layer	Shape	Activation
Linear	$(8+K) \rightarrow 64$	ReLU
Linear	64 $\rightarrow$ 64	ReLU
Mean head	64 $\rightarrow$ 1	—
Log-std head	64 $\rightarrow$ 1	Clamp $[-20, 2]$

The raw sample $z \sim \mathcal{N}(\mu, \sigma^2)$ is squashed via $a = \tanh(z) \times 60$ to produce a holding action in $[-60, 60]$ s. Negative values are interpreted as a request for early dispatch (clipped at the bus's earliest feasible time); the deployment-time effective range is $[0, 60]$ s. Log-probabilities include the standard tanh Jacobian correction [7]. Weights in the output heads are initialized uniformly in $[-3 \times 10^{-3}, 3 \times 10^{-3}]$. The same network is shared across all buses (parameter sharing), with bus-specific behavior arising from heterogeneous input states.

Reward Q-ensemble $\{Q_k\}_{k=1}^K$ (RE-SAC, $K=10$).

The lower replaces the standard twin-Q SAC critic by an ensemble of $K=10$ independently-initialised Q-networks; the policy is improved against a lower-confidence-bound (LCB) value $\bar{Q} + \beta_{\text{LCB}}\sigma_Q$ with $\beta_{\text{LCB}} = -2$, computed across the ensemble at every gradient step. Each ensemble member shares the same MLP shape:

Layer	Shape	Activation
Linear	$(8+K+1) \rightarrow 64$	ReLU
Linear	64 $\rightarrow$ 64	ReLU
Linear	64 $\rightarrow$ 1	—

Each Q_k takes the concatenation of the lower state and the scalar action as input. All members share a common mean target $(1/K) \sum_k \bar{Q}_k$ during temporal-difference learning to prevent ensemble divergence; per-Q MSE is augmented with a small ensemble-spread penalty $\beta_{\text{ood}} \text{std}_k(Q_k)$ ($\beta_{\text{ood}}=0.1$) that encourages disagreement on out-of-distribution actions. All ensemble members are accompanied by exponential moving average target networks with Polyak rate $\tau = 0.005$. (We use a Q-ensemble rather than a twin Q to obtain calibrated lower-confidence bounds; the twin-Q variant in standard SAC corresponds to $K=2$ without the LCB.)

Cost Q-network Q_c .

A single MLP with the same MLP shape as one ensemble member estimates the expected cumulative cost (headway deviation) under the current policy:

Layer	Shape	Activation
Linear	$(8+K+1) \rightarrow 64$	ReLU
Linear	$64 \rightarrow 64$	ReLU
Linear	$64 \rightarrow 1$	—

The cost critic is trained with the same target-network mechanism and soft update rate as the reward critics. Its output enters the policy loss through the Lagrangian multiplier λ : the policy minimizes $\alpha \log \pi - Q + \lambda Q_c$.

D Auxiliary hindsight credit: derivation and contrast with cross-advantage

The main cross-level coupling in TransitDuet is the goal channel of section 4.4: the upper outputs a per-dispatch target-headway shift that resets the lower’s Lagrangian target. This appendix derives the auxiliary hindsight credit on the upper reward (Eq. 23 in section 4.4), an upper-only signal that supports per-dispatch credit assignment, and contrasts it with a literal asynchronous extension of the cross-advantage objective of Pan et al. [5]. The literal cross-advantage extension corresponds to one of the architectural alternatives we evaluated end-to-end (Section 5.2); the goal-conditioned design we ultimately adopt sidesteps the issues described below by avoiding upper-advantage injection into the lower altogether.

D.1 Synchronous cross-advantage (RoboDuet, background)

In the synchronous setting of Pan et al. [5], two agents act at every time step t on a shared state s_t . The cross-advantage augmented objective is

$$\mathcal{L}_{\text{upper}}^{\text{sync}} = -\mathbb{E}_t[(A_U(s_t, a_t^U) + \beta A_L(s_t, a_t^L)) \nabla \log \pi_U(a_t^U | s_t)], \quad (30)$$

which is well-defined because both advantages are evaluated at the same shared state-action pair.

D.2 The asynchrony challenge

In TransitDuet, π_U acts at dispatch events $\{t_1^U, \dots, t_N^U\}$ while π_L acts at station arrivals $\{t_1^L, \dots, t_M^L\}$ with $M \gg N$, on disjoint event sets and different state spaces. There is no single time step at which A_U and A_L are jointly defined on a shared state, so the direct sum in Eq. (30) is undefined.

D.3 Trip-lifecycle alignment and the realised gap

We resolve this through a trip-lifecycle alignment. Each upper action $a_{U,i}$ at dispatch t_i^U initiates a trip τ_i launched at the baseline time $t_{\text{base},i}$ with a dispatch-specific target headway $h^{\text{target}} + \delta_{t,i}$. The realised inter-dispatch gap is the actual elapsed time between consecutive bus departures as carried out by the lower controller (which may hold individual buses to track its target):

$$g_i = t_{\text{actual},i+1} - t_{\text{actual},i}, \quad (31)$$

computed per direction so that buses operating on opposite legs do not interleave. The per-direction mean and standard deviation of $\{g_i\}$ define the deviation

$$\Delta_i = \frac{|g_i - \bar{g}|}{\sigma_g}. \quad (32)$$

D.4 Hindsight-credited upper reward

The upper reward at dispatch i is the sum of the episode-level system reward (a belief-weighted MORL scalarisation of wait, fleet overshoot, and headway CV; Eq. 28) and a per-dispatch hindsight credit derived from Δ_i :

$$R_U^{(i)} = R_{\text{sys}}(\mathbf{z}) - \alpha_H \cdot \frac{\Delta_i - \bar{\Delta}}{\sigma_{\Delta}}, \quad \alpha_H = 0.5. \quad (33)$$

The credit is recentred per episode so that dispatches with above-average gap deviation receive a negative credit and dispatches with below-average deviation receive a positive one; the system reward is shared across all dispatches in an episode.

D.5 Why we did not use literal cross-advantage

A literal asynchronous extension of Eq. (30) would aggregate per-trip lower advantages into the upper objective:

$$\tilde{R}_U^{\text{lit},(i)} = R_U^{(i)} + \beta \cdot \frac{1}{|\mathcal{J}(i)|} \sum_{j \in \mathcal{J}(i)} A_L(s_{L,j}, a_{L,j}), \quad (34)$$

where $\mathcal{J}(i)$ is the set of lower-level transitions during trip i . We tried this in early prototypes and found it unstable. Two diagnostics: (i) the per-step lower advantage and the per-dispatch upper advantage differ in scale by 1–2 orders of magnitude across a trip lifecycle, so a single scalar β either suppresses cross-level signal or destabilises lower training; (ii) the lower’s advantage estimator uses an LCB-discounted ensemble Q (section 4.3), which is a non-linear function of the policy parameters, breaking the simple unbiasedness argument that holds in the synchronous setting. The hindsight-credit formulation in Eq. (33) is a coarser but markedly more stable alternative: it injects per-trip execution outcome (gap regularity) into upper credit assignment without requiring lower advantage backflow.

D.6 What changes for our off-policy SAC upper

The TransitDuet upper is trained as RE-SAC (section 4.2, Eqs. 12–13), not as REINFORCE: each per-dispatch transition is pushed to a replay buffer, and the policy/critic are updated by repeated mini-batch SAC gradient steps. The REINFORCE-based unbiasedness argument that applies to a synchronous on-policy estimator is therefore not the right analytical lens for our setup. In practice the lower is also being updated between upper SAC gradient steps, so the upper’s training distribution is non-stationary; the goal-conditioned coupling (section 4.4) and TPC’s importance reweighting (section 4.8) together control this non-stationarity at the lower side, and the upper’s ensemble-LCB Q value provides conservatism against epistemic uncertainty in the upper’s own off-policy training.

References

- [1] GKIOTSALITIS K, CATS O. At-stop control measures in public transport: Literature review and research agenda[J/OL]. Transportation Research Part E: Logistics and Transportation Review, 2021, 145: 102176. DOI: [10.1016/j.tre.2020.102176](https://doi.org/10.1016/j.tre.2020.102176).
- [2] WANG J, SUN L. Dynamic holding control to avoid bus bunching: A multi-agent deep reinforcement learning framework[J/OL]. Transportation Research Part C: Emerging Technologies, 2020, 116: 102661. DOI: [10.1016/j.trc.2020.102661](https://doi.org/10.1016/j.trc.2020.102661).
- [3] SUTTON R S, PRECUP D, SINGH S. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning[J/OL]. Artificial Intelligence, 1999, 112(1–2): 181–211. DOI: [10.1016/S0004-3702\(99\)00052-1](https://doi.org/10.1016/S0004-3702(99)00052-1).
- [4] NACHUM O, GU S, LEE H, et al. Data-efficient hierarchical reinforcement learning[C]// Advances in Neural Information Processing Systems: volume 31. 2018: 3303–3313.

- [5] PAN G, BEN Q, YUAN Z, et al. RoboDuet: Learning a cooperative policy for whole-body legged loco-manipulation[J/OL]. IEEE Robotics and Automation Letters, 2025, 10(5): 4564-4571. DOI: [10.1109/LRA.2025.3551230](https://doi.org/10.1109/LRA.2025.3551230).
- [6] TESSLER C, MANKOWITZ D J, MANNOR S. Reward constrained policy optimization[C]// Proceedings of the 7th International Conference on Learning Representations. 2019.
- [7] HAARNOJA T, ZHOU A, ABBEEL P, et al. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor[C]//Proceedings of Machine Learning Research: volume 80 Proceedings of the 35th International Conference on Machine Learning. PMLR, 2018: 1861-1870.
- [8] MIRYOOSEFI S, BRANTLEY K, DAUMÉ III H, et al. Reinforcement learning with convex constraints[C]//Advances in Neural Information Processing Systems: volume 32. 2019: 14093-14102.
- [9] LI S, WANG R, TANG M, et al. Hierarchical reinforcement learning with advantage-based auxiliary rewards[C]//Advances in Neural Information Processing Systems: volume 32. 2019: 1409-1419.
- [10] SINGH U, SUTTLE W A, SADLER B M, et al. PIPER: Primitive-informed preference-based hierarchical reinforcement learning via hindsight relabeling[A]. 2024. arXiv: [2404.13423](https://arxiv.org/abs/2404.13423).
- [11] ALESIANI F, GKIOTSALITIS K. Reinforcement learning-based bus holding for high-frequency services[C/OL]//Proceedings of the 21st IEEE International Conference on Intelligent Transportation Systems (ITSC). 2018: 3162-3168. DOI: [10.1109/ITSC.2018.8569473](https://doi.org/10.1109/ITSC.2018.8569473).
- [12] ZHANG Y, ZHENG L. RE-SAC: Disentangling aleatoric and epistemic risks in bus fleet control: A stable and robust ensemble DRL approach[A]. 2026. arXiv: [2603.18396](https://arxiv.org/abs/2603.18396).
- [13] IBARRA-ROJAS O J, DELGADO F, GIESEN R, et al. Planning, operation, and control of bus transport systems: A literature review[J/OL]. Transportation Research Part B: Methodological, 2015, 77: 38-75. DOI: [10.1016/j.trb.2015.03.002](https://doi.org/10.1016/j.trb.2015.03.002).
- [14] CATS O, GLÜCK S. Frequency and vehicle capacity determination using a dynamic transit assignment model[J/OL]. Transportation Research Record: Journal of the Transportation Research Board, 2019, 2673(3): 574-585. DOI: [10.1177/0361198118822292](https://doi.org/10.1177/0361198118822292).
- [15] VEZHNEVETS A S, OSINDERO S, SCHAUL T, et al. FeUdal networks for hierarchical reinforcement learning[C]//Proceedings of Machine Learning Research: volume 70 Proceedings of the 34th International Conference on Machine Learning. PMLR, 2017: 3540-3549.
- [16] LEE J, LEE B J, KIM K E. Reinforcement learning for control with multiple frequencies[C]// Advances in Neural Information Processing Systems: volume 33. 2020: 3254-3264.
- [17] METELLI A M, MAZZOLINI F, BISI L, et al. Control frequency adaptation via action persistence in batch reinforcement learning[C]//Proceedings of Machine Learning Research: volume 119 Proceedings of the 37th International Conference on Machine Learning. PMLR, 2020: 6862-6873.
- [18] WATANABE K, STRONG M, ELDAR O. SHIRO: Soft hierarchical reinforcement learning[A]. 2022. arXiv: [2212.12786](https://arxiv.org/abs/2212.12786).
- [19] STOOKE A, ACHIAM J, ABBEEL P. Responsive safety in reinforcement learning by PID Lagrangian methods[C]//Proceedings of Machine Learning Research: volume 119 Proceedings of the 37th International Conference on Machine Learning. PMLR, 2020: 9133-9143.
- [20] DAYAN P, HINTON G E. Feudal reinforcement learning[C]//Advances in Neural Information Processing Systems: volume 5. 1993: 271-278.